

Traitement d'image embarqué pour la détection de coin

Ce rapport de validation démontre l'implémentation des spécifications du cahier des charges du projet de traitement d'image embarqué pour la détection de coin. La plateforme Vivado sera utilisé pour concevoir le projet implémenté sur une carte CoraZ7. L'objectif final est de créer une chaîne de traitement permettant de mettre en évidence les coins d'une image en appliquant différents filtres.

Les livrables sont constitués du plan et du rapport de validation, des schémas d'architectures et du schéma synoptique global du projet ainsi que des codes sources.

Un projet Vivado à été créée par IP. Le bloc design général est amendé à chaque étape avec l'IP validé.

Sommaire

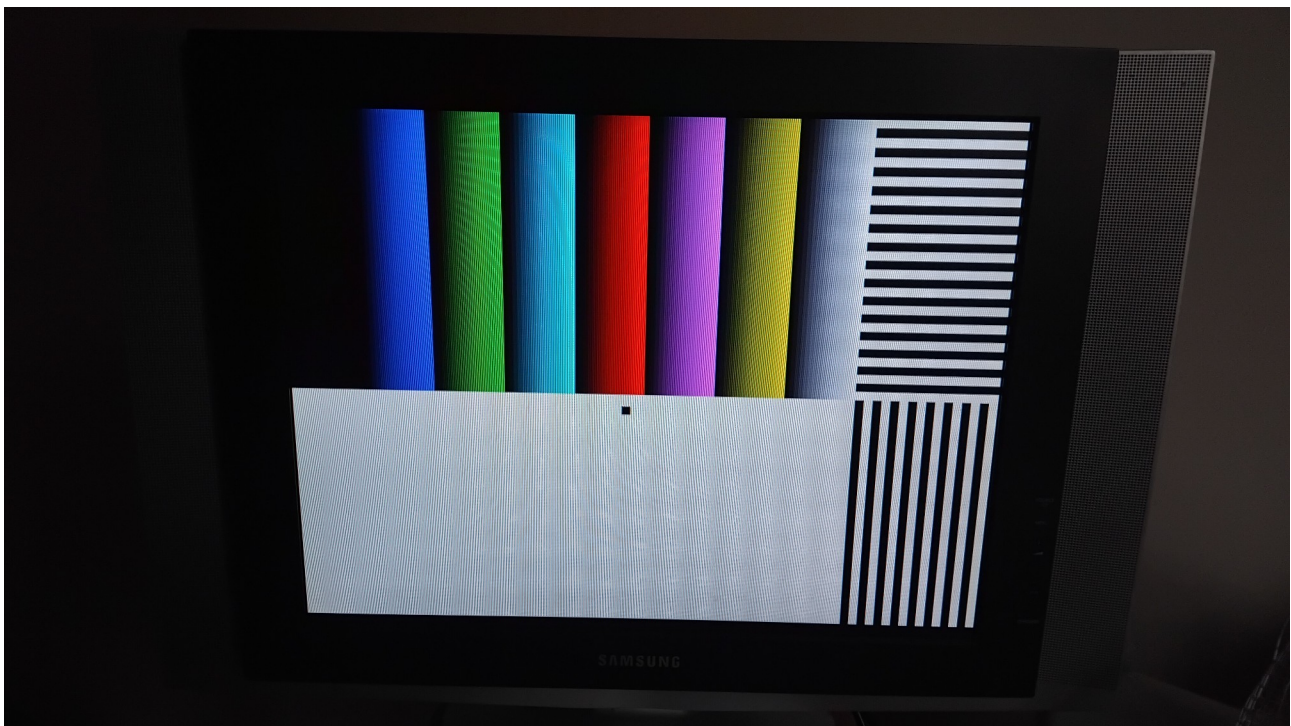
	Nom de l'étape	Validation
1.X	Affichage d'un Flux Vidéo	
1.1	Vérification bon fonctionnement écran	✓
1.2	Validation bloc de pilotage Écran	✓
2.1	Validation du Test Pattern generator	✓
2.X	Affichage d'une image stockée dans la mémoire.	
2.1	Validation du bloc de contrôle Axis-reg.	✓
2.2	Validation du DMA.	✓
3.X	Chaîne de traitement	
3.1	Validation du sélecteur de flux	✓
3.2	Validation du bloc de conversion luminance	✓
3.3	Validation du Filtre Gaussien	X
3.4	Validation du Filtre Harris	X
3.5	Validation du bloc de Seuillage	X
3.6	Validation de l'Overlay	X
4.X	Fonctionnalités supplémentaires	
4.1	Ajout de la remontée d'information depuis bloc de seuillage.	X
4.2	Lecture d'une image stockée dans la carte USB.	X

1.X : Affichage d'un flux vidéo

1.1 : Vérification du bon fonctionnement de l'écran

Test 1.1.1 : Le test est réussi si le code de référence fourni par les fabricants Pmod VGA permet l'affichage sur l'écran de l'image de test.

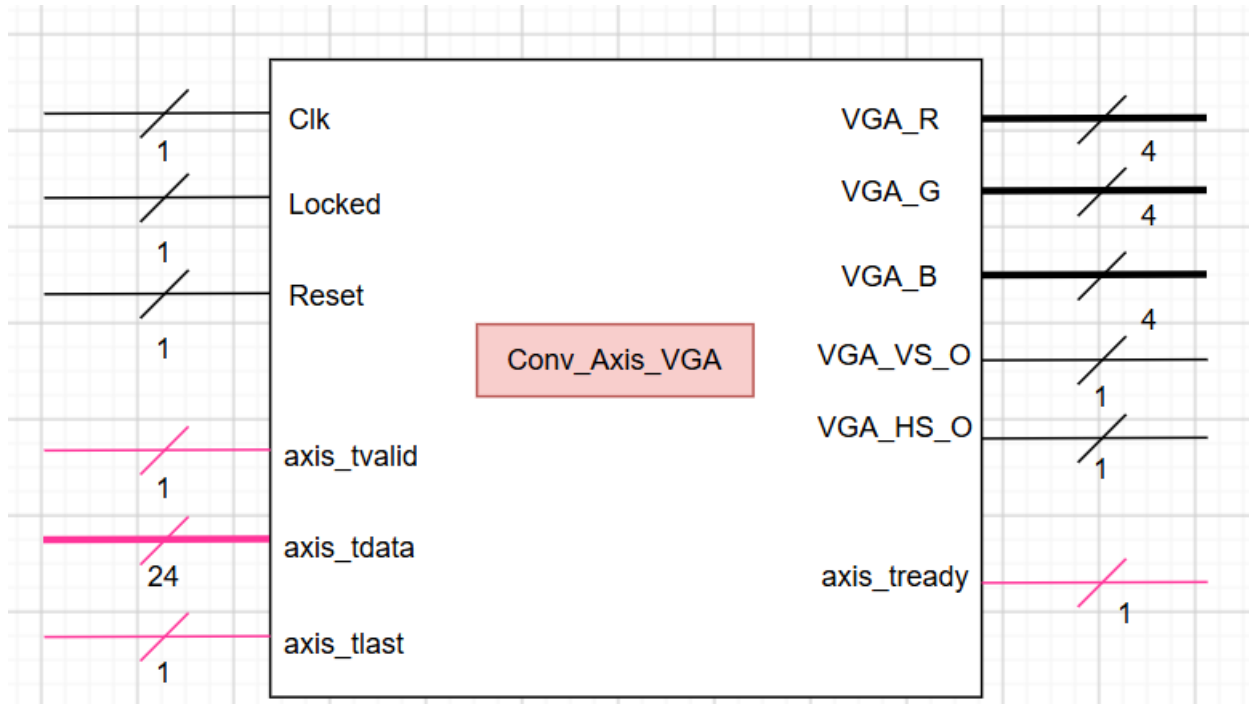
Le matériel fourni comprend un adaptateur qui se connecte sur la carte Cora avec une sortie VGA, cette étape de validation consiste à vérifier que l'écran, le câble et l'adaptateur fonctionne correctement. Une fois l'affichage validé, il sera possible de l'utiliser pour les étapes suivantes.



On constate que l'image de test est correctement affichée sur l'écran. Il n'y a pas de distorsion chromatique, un bloc de pixel noir se déplace sur l'écran. Le test est validé.

1.2 Validation du bloc de pilotage Écran

1) Description fonctionnelle du bloc



Cet élément prend en entrée un bus à la norme AXI4-S. Ce bus vaut 24 bits car on transporte les trois canaux de pixel = 8*3 et pilote la sortie VGA Trois Canaux RGB, H sync et V sync. Chaque canal d'entrée est sur **8 bits** car il s'agit de la valeur d'intensité des pixels sur 256 valeurs. Les sorties VGA sont sur **4 bits** (16 nuances de couleur) car il sortent sur le CNA permettant de retrouver une valeur analogique à l'aide de ponts diviseur de tension. (cf Annexe note sur la norme VGA).

Vect_input : 8 bits d'entrée

Vect_output : 4 bits de sortie.

$$Vect_{out} = \frac{Vect_{input} \times 15}{255}$$

2) Description des signaux internes et externes

Signaux externe à l'IP		
Nom du signal	Type	Description
AXIS_TDATA_WIDTH	Generic	Variable permettant de fixer la taille du vecteur de données T_data, vaut normalement 24 bits.

Clk	Input	Horloge à 25 Mhz.
locked	Input	Locked vaut 1 lorsque l'horloge en sortie de la PLL est stable.
resetsn	Input	Actif à l'état bas.
TVALID	Input	Vaut 1 lorsque les données reçues sont correctes.
TREADY	Output	Ready vaut à 1 lorsque on est prêt à recevoir des données Ready est passé à 0 lorsque les compteurs sortent de la zone utile de l'image.
TDATA	Input	Vecteur 24 bits concaténant les 3 vecteurs 8 bits RGB.
TLAST	Input	Tlast indique une fin de transmission
VGA_VS_O	Output	Signal de synchronisation Vertical cf norme VGA
VGA_HS_O	Output	Signal de synchronisation Horizontal cf norme VGA
VGA_R	Output	Signal 4 bit (valeur entre 0 et 15) servant à piloter le driver VGA canal rouge.
VGA_G	Output	Signal 4 bit servant à piloter le driver VGA canal Vert
VGA_B	Output	Signal 4 bit servant à piloter le driver VGA canal Bleu.
Signaux interne à l'IP		
Nom du signal	Type	Description
FRAME_WIDTH	Natural	Constante renseignant sur la largeur de l'écran
FRAME_HEIGHT	Natural	Constante indiquant la hauteur de l'écran
H_FP	Natural	Front porch horizontal
H_PW	Natural	Largeur de pulse horizontal
H_MAX	Natural	Nombre de pixel total sur une ligne (image visible + virtuelle)
V_FP	Natural	Front porch vertical
V_PW	Natural	Largeur de pulse vertical
V_MAX	Natural	Nombre total de lignes (image visible + image virtuelle)
H_POL	std_logic	Polarité du pulse de synchronisation horizontal
V_POL	std_logic	Polarité du pulse de synchronisation vertical
S_VGA_R	std_logic_vector	Signal recevant la conversion 8 → 4 bits pour le pilotage des pixels rouges
S_VGA_B	std_logic_vector	Signal recevant la conversion 8 → 4 bits pour le pilotage des pixels bleus

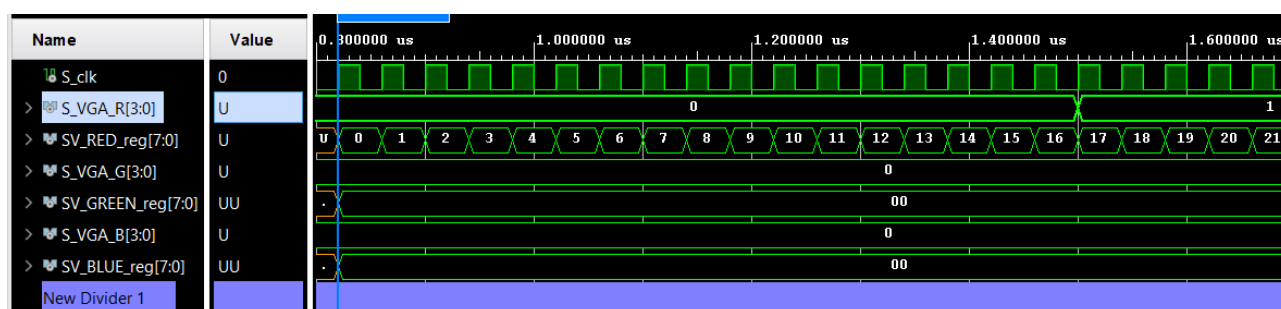
S_VGA_G	std_logic_vector	Signal recevant la conversion 8 → 4 bits pour le pilotage des pixels verts
S_VGA_VS_O	std_logic	Signal interne de synchronisation verticale
S_VGA_HS_O	std_logic	Signal interne de synchronisation horizontale
S_CTRL_H	std_logic_vector	Compteur permettant de se repérer horizontalement sur l'écran.
S_CTRL_V	std_logic_vector	Compteur permettant de se repérer verticalement sur l'écran.
S_AXIS_ACLK	std_logic	Signal interne d'horloge
S_AXIS_ARESETN	std_logic	Signal interne recevant le le signal axis correspondant
S_AXIS_TREADY	std_logic	Signal interne recevant le le signal axis correspondant
S_AXIS_TDATA	std_logic_vector	Signal interne recevant le le signal axis correspondant
S_AXIS_TLAST	std_logic	Signal interne recevant le le signal axis correspondant
S_AXIS_TVALID	std_logic	Signal interne recevant le le signal axis correspondant
SV_RED_reg	std_logic_vector	Vecteur récupérant la partie rouge de Tdata. (8 bits LSB)
SV_GREEN_reg	std_logic_vector	Vecteur récupérant la partie verte de Tdata. (15 - 8)
SV_BLUE_reg	std_logic_vector	Vecteur récupérant la partie bleue de Tdata. (8 MSB)
SV_vga_red_out_reg	std_logic_vector	Registre de temporisation désactivé pour le moment
SV_vga_green_out_reg	std_logic_vector	Registre de temporisation désactivé pour le moment
SV_vga_blue_out_reg	std_logic_vector	Registre de temporisation désactivé pour le moment
SV_Vsync_out_reg	std_logic	Registre de temporisation désactivé pour le moment
SV_Vsync_out_reg	std_logic	Registre de temporisation désactivé pour le moment

3) Test et validation du bloc

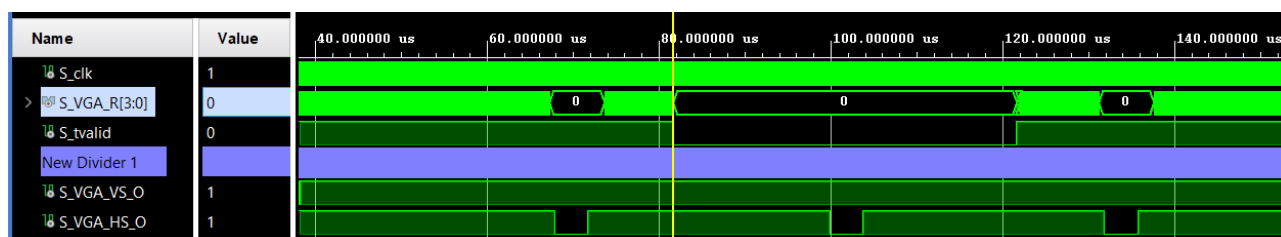
Test 1.2.1 : La brique développée est testée à l'aide d'un Test bench et d'un flux contrôlé ; le vecteur de données T_data est incrémenté à chaque coup d'horloge.

Signaux	Condition de validation	Réussi ?
VGA_R, VGA_G, VGA_B.	Les signaux de sortie sont entre 0 et 15 et dépendent de l'entrée TDATA. TDATA prend des valeurs entre 0 et 255, cette amplitude est ramenée entre 0 et 15 de manière proportionnelle. Lorsque TVALID vaut 0, les pixels diffusés sont noirs et les compteurs	

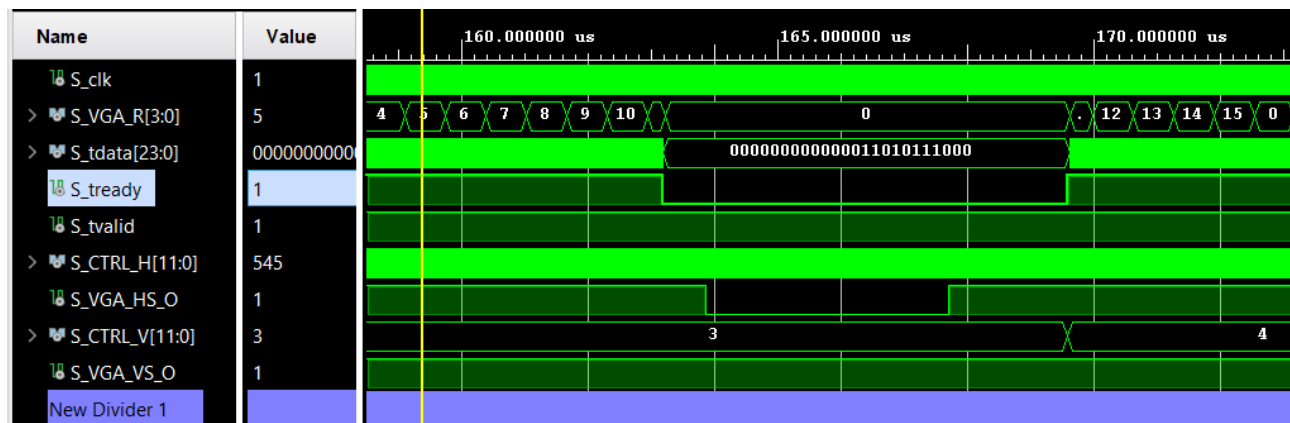
	internes continuent à s'incrémenter.	
VGA_VS_O	La taille du pulse est égal à 64 us (2*800 pixels) Le délais entre deux pulses est égal à 16,8 ms (525*800 pixels)	
VGA_HS_O	La taille du pulse est égal à 3,84 us (96 pixels) Le délais entre deux pulses est égal à 32 us (800 pixels)	
TREADY	Tready vaut 1 lorsque les pixels diffusés sont dans l'écran utile (Hors image virtuelle)	
TRESET	Lorsque TRESET est actif, les sorties VGA et les compteurs de lignes/colonnes sont mises à 0. Treset est actif sur locked actif.	



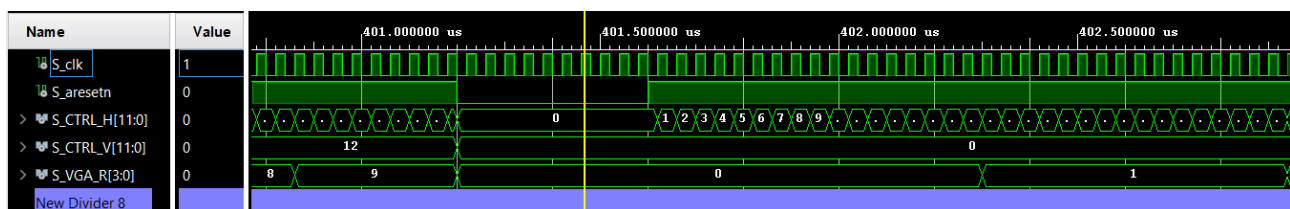
S_VGA_R récupère le produit de l'opération des registres d'entrée SV_RED_reg, S_VGA est actualisé avec un retard de un coup d'horloge par rapport à SV_RED_reg. Le chronogramme ci-dessus indique que S_VGA_R vaut 0 lorsque SV_RED_reg est entre 0 et 15. Le découpage est régulier et conforme au résultat attendu.



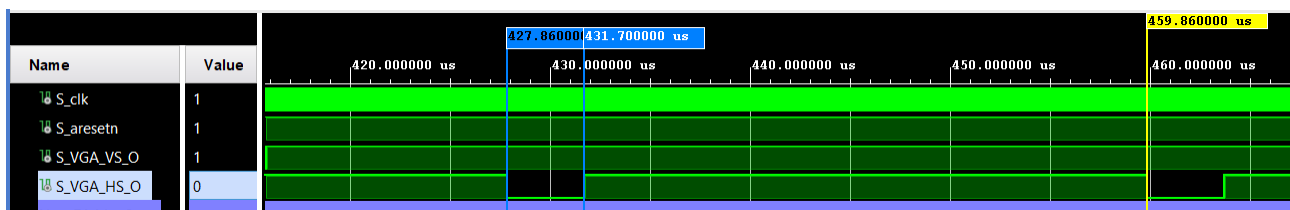
Lorsque S_tvalid est à l'état bas, les pixels débités sur les trois canaux sont noirs mais le compteur permettant de se localiser sur l'écran est toujours actif.



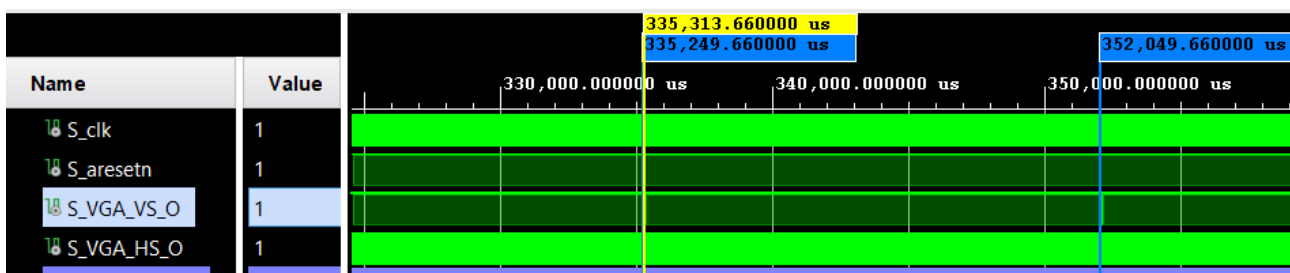
Lorsque S_CTRL_H est supérieur à 639, S_Tready tombe et le flux incident de données arrête de s'incrémenter.



Lorsque S_arestn passe à 0, S_CTRL_H et S_CTRL_V sont réinitialisés ainsi que les sorties VGA.



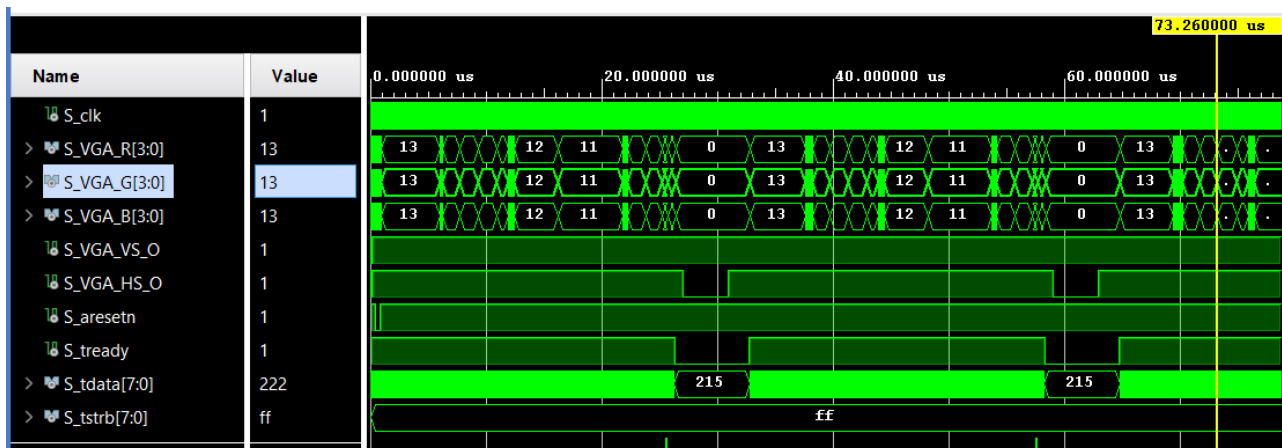
Le signal de synchronisation horizontale à une largeur de pulse de 3,84 us,
Le délais entre deux pulses est égal à 32us.



Le signal de synchronisation Vertical à une largeur de pulse de 64 us,
Le délais entre deux pulses est égal à 16 800us.

Test 1.2.2 : La brique sera ensuite validée avec le lecteur de fichier fourni par les formateurs.

On simule maintenant le Convertisseur Axis VGA avec la brique lisant un fichier fournit par Deverne.



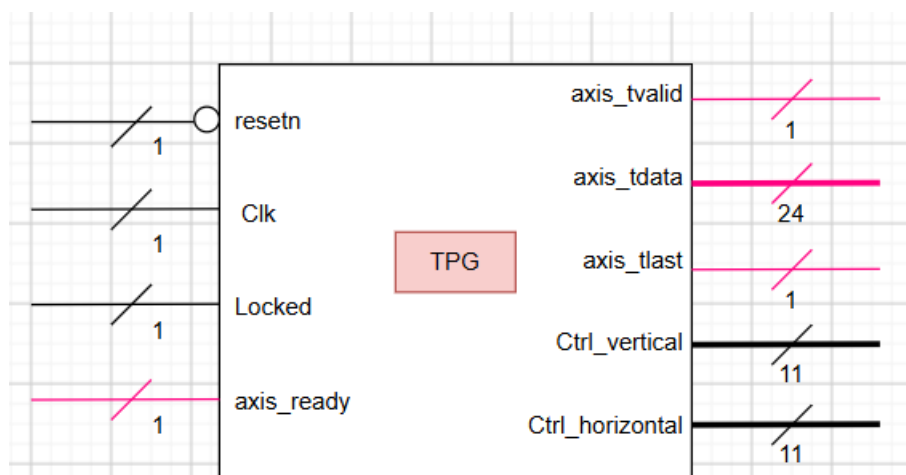
Simulation avec le lecteur de fichier fournit.

S_ready interrompt le transfert de donnée lorsqu'il vaut 0, le front et back porch sont remplis avec des pixels noirs.

Tout les signaux VGA sont égaux retranscrivant l'image en niveaux de gris.

2.1 : Écriture du Test Pattern generator

1) Description fonctionnelle du bloc



Le TPG débite des données lorsque la valeur de T_ready vaut 1. Lorsque T_ready est à l'état bas les compteurs internes au TPG ne s'incrémentent plus. Le TPG est conçu pour émettre une mire constituée d'un bloc de pixel blanc sur fond noir qui se déplace sur l'écran.

2) Description des signaux externes et internes

Signaux externe à l'IP		
Nom du signal	Type	Description
AXIS_TDATA_WIDTH	Generic	Défini la taille du vecteur data = 24 en RGB
Clk	IN	Signal d'horloge
Locked	IN	Locked renseigne sur la stabilité de l'horloge. Si locked = 1, L'horloge est stable.
resetn	IN	Signal de remise à zéro asynchrone.
axis_tready	IN	Signal indiquant que les données sont prête à être envoyé, dans notre cas, elles sont toujours prête. Tready = 1
axis_tdata	OUT	Vecteur de données RGB le rouge occupe les 8 LSB et le Bleu les MSB
<i>axis_tstrb</i>	<i>OUT</i>	<i>Vecteur de sélection des données à modifier, dans notre cas toujours à 1</i>
axis_tlast	OUT	Pulse de fin de ligne.
axis_tvalid	OUT	Signal indiquant que le recepteur est prêt à recevoir le stream de données.
Ctrl_vertical	OUT	Compteur de ligne
Ctrl_horizontal	OUT	Compteur s'incrémentant à chaque pixel débité, réinitialisé à chaque ligne.
Signaux interne à l'IP		
Nom du signal	Type	Description
SV_red_out	std_logic_vector	Vecteur permettant d'extraire du flux de données AXIS (Tdata) la valeur du rouge.
SV_green_out	std_logic_vector	Vecteur permettant d'extraire du flux de données AXIS (Tdata) la valeur du vert.
SV_blue_out	std_logic_vector	Vecteur permettant d'extraire du flux de données AXIS (Tdata) la valeur du bleue.
s_clk	std_logic	Signal de récupération de l'horloge.
h_cntr_reg	std_logic_vector	Registre de contrôle horizontal est compris entre 0 et 639. C'est un compteur qui permet de savoir quel pixel de la ligne est envoyé.
v_cntr_reg	std_logic_vector	Registre de contrôle vertical. Est compris entre 0 et 479.
FRAME_WIDTH	natural	Largeur de l'écran : 640 pixels

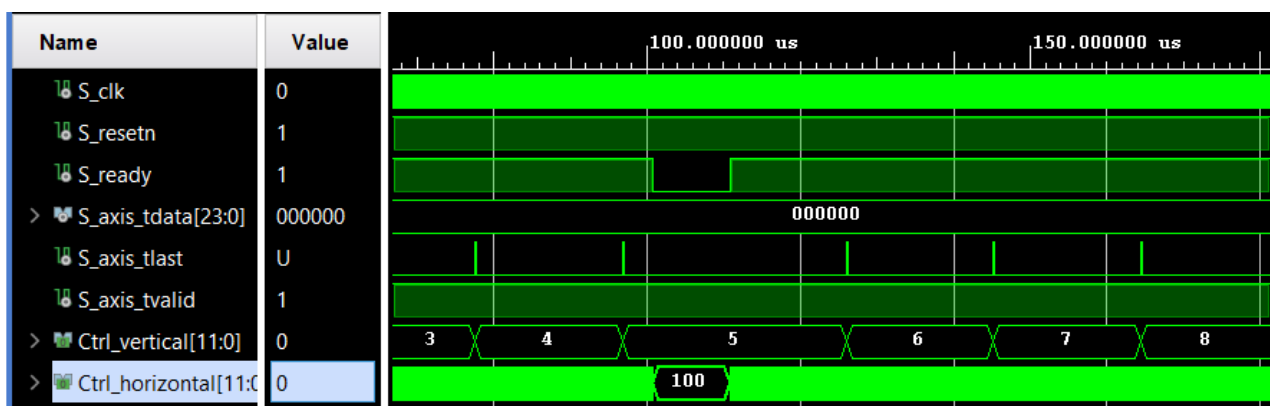
FRAME_HEIGHT	natural	Hauteur de l'écran : 480 pixels
BOX_WIDTH	natural	Taille du carré mouvant.
BOX_CLK_DIV	natural	Diviseur d'horloge pour ralentir l'actualisation de la position du carré mouvant.
BOX_X_MAX	natural	Borne supérieur horizontale délimitant la zone où le carré mouvant peut se déplacer.
BOX_Y_MAX	natural	Borne supérieur verticale délimitant la zone où le carré mouvant peut se déplacer.
BOX_X_MIN	natural	Borne inférieure horizontale délimitant la zone où le carré mouvant peut se déplacer.
BOX_Y_MIN	natural	Borne inférieure verticale délimitant la zone où le carré mouvant peut se déplacer.
BOX_X_INIT	std_logic_vector	Position initiale horizontale du carré mouvant à l'allumage
BOX_Y_INIT	std_logic_vector	Position initiale verticale du carré mouvant à l'allumage
s_box_x_reg	std_logic_vector	Registre mémorisant la position horizontale du carré mouvant
s_box_x_dir	std_logic	Sens de déplacement horizontale du carré mouvant.
s_box_y_reg	std_logic_vector	Registre mémorisant la position verticale du carré mouvant
s_box_y_dir	std_logic	Sens de déplacement verticale du carré mouvant.
s_box_cntr_reg	std_logic_vector	compteur cadencé sur l'horloge 25Mhz permettant l'actualisation de l'emplacement du carré mouvant.
s_update_box	std_logic	S_update_box vaut 1, la position et la direction du carré est actualisé.
s_pixel_in_box	std_logic	Signal indiquant que le pixel sortant est dans la boîte et qu'il faut donc lui appliquer une couleur.
S_resetrn	std_logic	Signal réceptionnant le resetrn.
S_axis_tready	std_logic	Signal réceptionnant le tready
S_axis_tstrb	std_logic_vector	<i>Toutes les données sortantes sont conservées, toujours égal à 1</i>
S_axis_tlast	std_logic	Tlast indique une fin de ligne, ce signal est indicatif.
S_axis_tvalid	std_logic	Tvalid vaut toujours 1 sauf lorsque reset ou locked est actif

3) Validation en simulation

Test : 2.1.1 : Validation à l'aide d'un TestBench

- Le signal Tvalid est à l'état haut lorsque resetn est inactif.
- Le signal Tlast est à l'état haut à la fin d'une ligne.
- si l'observation du signal Tdata permet d'identifier qu'un bloc de 8x8 pixels est bien généré.

Signaux	Condition de validation	Réussi
resetn	L'état actif est conditionné par le signal locked. h_cntr_reg & c_cntr_reg = 0. s_axis_tready = 0 s_box_x_reg = 0 & s_box_y_reg = 400 s_box_x_dir & s_box_y_dir = 1 s_box_cntr_reg = 0	
axis_tvalid	Sur axis_tvalid = 0, les données ne sont plus débitées. Les compteurs internes arrêtent de s'incrémenter.	
axis_tdata	Il est difficile de contrôler ce qui est envoyé en simulation, mais le bloc a été programmée pour envoyer un carré de 8x8 pixels blancs sur fond noir. Il est attendu que axis_tdata soit majoritairement égal à 0 et passe à 1 à intervalle régulier sur 8 coups d'horloges.	
Ctrl_vertical et Ctrl_horizontal	Ctrl_horizontal s'incrémente à chaque pixel débité et est réinitialisé à chaque ligne complète (image visible uniquement). Ctrl Vertical s'incrémente à chaque nouvelle ligne et est réinitialisé une fois que toute l'image a été balayée (image visible uniquement).	
axis_tlast	Tlast est généré à chaque fin de ligne.	



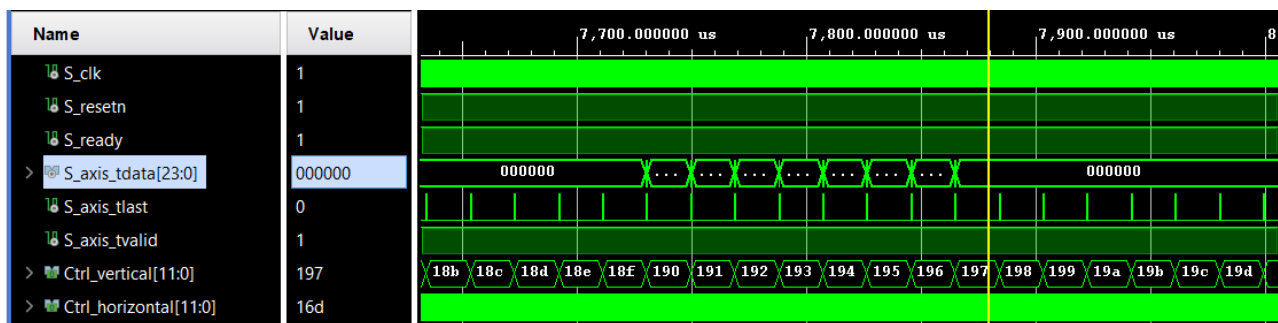
Validation du bon fonctionnement de tlast, tready sur le bloc TPG

Ctrl_vertical et ctrl_horizontal se figent lorsque T_ready est à l'état bas, de plus Tlast dur 1 coup d'horloge et est émis à chaque fin de ligne. Ctrl_vertical d'incrémente à chaque fin de ligne.



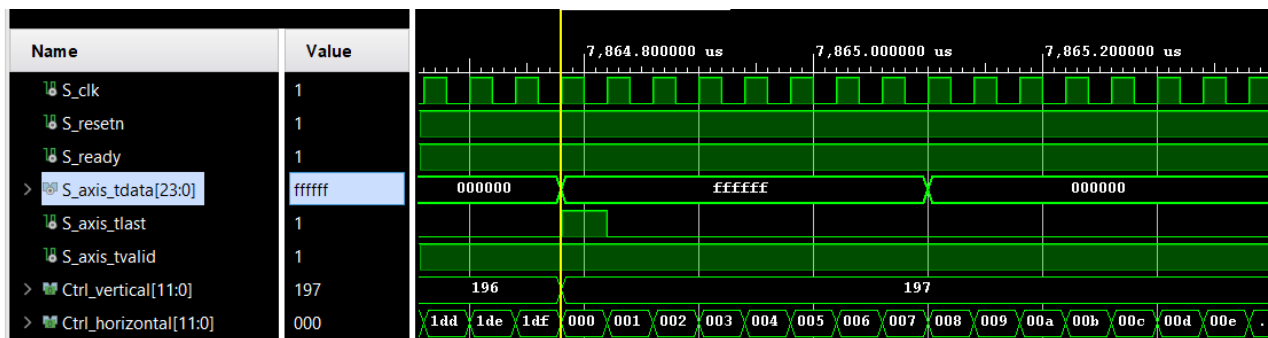
Validation bon fonctionnement reset.

Sur resetrn actif, les registres de comptage h_cntr_reg, v_cntr_reg et s_box_cntr_reg sont réinitialisés à 0.



Validation bon fonctionnement tdata.

Tdata est principalement à 0 sauf sur 8 lignes où le vecteur est mis à 1 pendant 8 coup d'horloge, cela indique que l'intensité des pixels R, G et B est maximale sur 8 pixels.

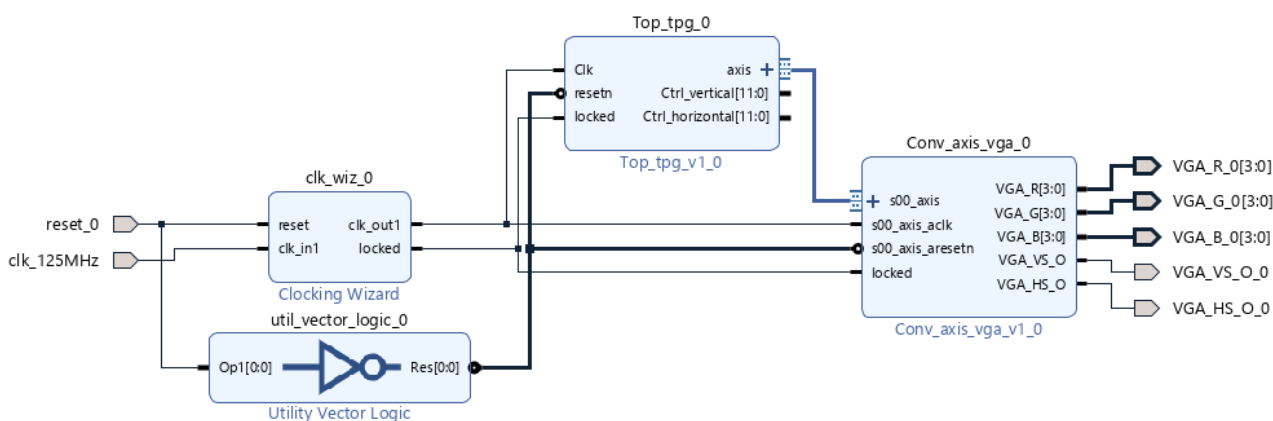


Validation bon fonctionnement tdata.

On observe que tdata vaut 1 pendant les 8 premiers pixels de la ligne, conformément à la configuration indiquant que la mire est constituée d'un carré de 8x8 pixels.

Test : 2.1.2 : **Démonstration** : Association du TPG et du bloc de pilotage de l'écran

Les deux blocs sont cascades et connectés à une PLL pour la génération de l'horloge à 25Mhz.



Cette étape de la validation permet de mettre en évidence que les conditions de réinitialisation entre les compteurs horizontaux et verticaux avaient été inversés.

Après correction, on obtient bien un carré de 8x8 pixels qui rebondit sur les bords de l'écran.

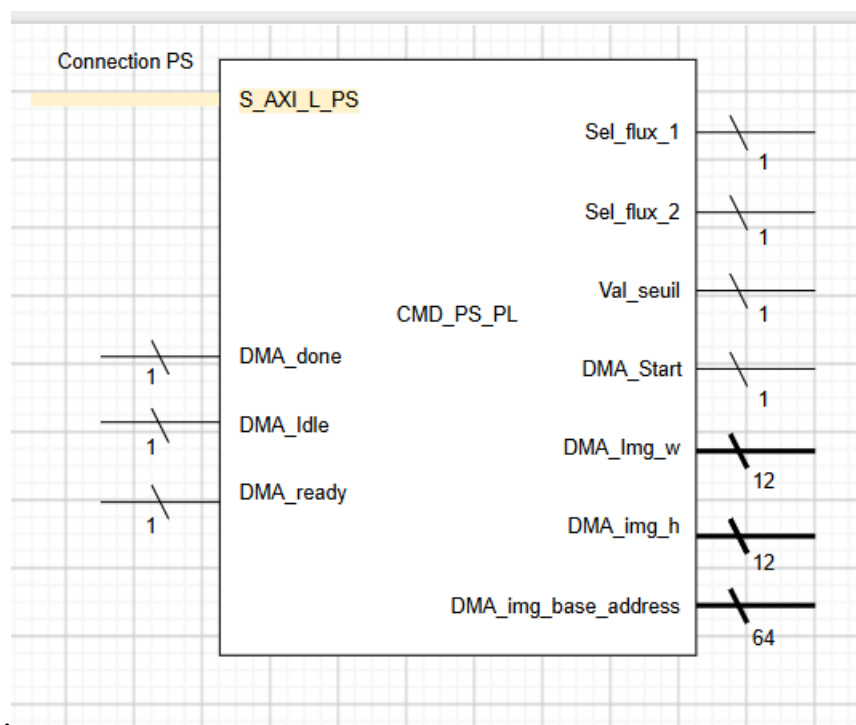


Validation de l'implémentation du TPG et du Convertisseur Axis VGA.

3. X Affichage d'une image stockée dans la mémoire.

3.1 Validation du bloc de contrôle Axis-reg.

1) Description fonctionnelle du bloc



Le bloc de commande permet de discuter avec la PS à l'aide d'une interface AXI4-Lite, il donne à d'autres bloc les informations suivantes à l'aide de registres internes :

- Le choix du flux pour les sélectionneurs de flux. Booléen 0 ou 1 car les sélecteurs de flux aurons deux flux d'entrée.
- La valeur du seuil pour le bloc de seuillage, valeur comprise entre 0 et 255.
- Le signal de démarrage du DMA ainsi que la largeur, hauteur et l'adresse de base de l'image situé dans la DDR.

2) Description des signaux internes et externes

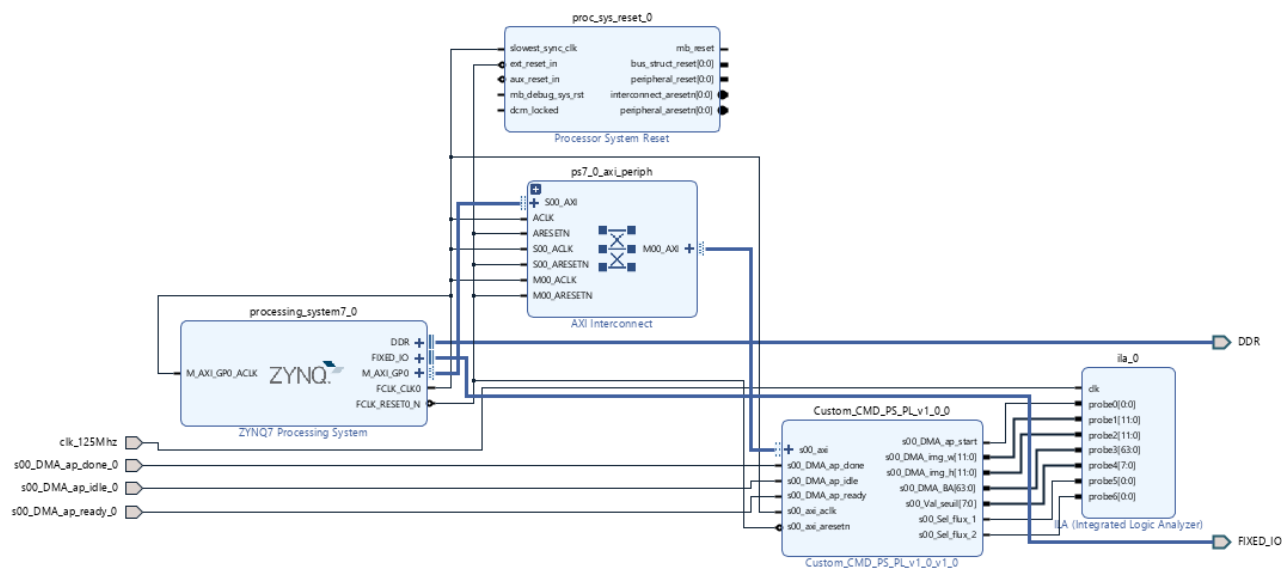
Description signaux Generique				
Nom		Type	Description	
C_S_AXI_DATA_WIDTH		Integer	Taille du bus de données (32bit) pour la liaison Axis avec le PS	
C_S_AXI_ADDR_WIDTH		Integer	Nombre de bits pour l’adressage.	
Description signaux externes				
Nom		Input/ Output	Type	Description
Bus AXI4-Lite				Bus Slave de communication avec le PS.
S_AXI_ACLK		Input	Std_logic	Horloge du bus
S_AXI_ARESETN		Input	Std_logic	Reset du bus
Canal AW	AWADDR	Input	Std_logic_vector (3 down to 0)	Base address : offset address pour l’écriture :
	AWPROT	Input		Niveau de protection. Indifférent pour nous.
	AWVALID	Input	Std_logic	Indique que l’adresse est prête à être envoyée
	AWREADY	Output	Std_logic	Indique que l’adresse est prête à être reçue
Canal data W	WDATA	Input	Std_logic_vector (63 down to 0)	Données brutes à écrire
	WSTRB	Input	Std_logic_vector (63 down to 0)	Indique quels données du bus sont à écrire (Toujours à 1?)
	WVALID	Input	Std_logic	Données prêtes à être envoyées.
	WREADY	Output	Std_logic	Bloc prêt à recevoir.
Canal W resp	BRESP	Output	Std_logic(1 downto 0)	Statut de la transaction d’écriture (Okay,Exokay, SLVERR,DECERR)
	BVALID	Output	Std_logic	Réponse d’écriture prêt à être envoyée
	BREADY	Input	Std_logic	Maitre prêt à recevoir
Canal Read Ad	ARADDR	Input	Std_logic_vector (3 down to 0)	
	ARPROT	Input		Niveau de protection. Indifférent pour nous.
	ARVALID	Input	Std_logic	Indique que le maitre à mis à disposition l’adresse à lire
	ARREADY	Output	Std_logic	Le slave est prêt à émettre l’adresse.
Canal lecture	RDATA	Output	Std_logic_vector (63 down to 0)	Valeur de la données à émettre
	RRESP	Output		Statut de la lecture.

	RVALID	Output	Std_logic	Données de lecture prête à être envoyé.
	RREADY	Input	Std_logic	Maitre prêt à lire.
DMA_img_w		Output	Std_logic_vector (11 downto 0)	Dimension de l'image (LARGEUR)
DMA_img_h		Output	Std_logic_vector (11 downto 0)	Dimension de l'image (HAUTEUR)
DMA_img_base_address		Output	Std_logic_vector (63 downto 0)	Base address (64 bit) de l'image dans la mémoire DDR
DMA_ap_Start		Output	Std_logic	Signal de contrôle du DMA.
DMA_done		Input	Std_logic	Signal d'état du DMA
DMA_idle		Input	Std_logic	Signal d'état du DMA
DMA_ready		Input	Std_logic	Signal d'état du DMA
Switch_stream_1		Output	Std_logic	Booléen indiquant quel stream le sélecteur de stream doit choisir.
Switch_stream_2		Output	Std_logic	Booléen indiquant quel stream le sélecteur de stream doit choisir.
Val_seuil		Output	Std_logic	Vecteur indiquant quel valeur de seuil le bloc de seuillage doit appliquer. [0 - 255]
Signaux internes				
NOM	Type		Commentaire	
slv_reg_0	STD_logic_vector de 32 bit		DMA Start Register	
slv_reg_1	STD_logic_vector de 32 bit		DMA Image W Register	
slv_reg_2	STD_logic_vector de 32 bit		DMA Image H Register	
slv_reg_3	STD_logic_vector de 32 bit		DMA BA Register LSB	
slv_reg_4	STD_logic_vector de 32 bit		DMA BA Register MSB	
slv_reg_5	STD_logic_vector de 32 bit		SEUIL VAL seuil register	
slv_reg_6	STD_logic_vector de 32 bit		Mux SEL FLUX 1	
slv_reg_7	STD_logic_vector de 32 bit		Mux SEL FLUX 2	
slv_reg_8	STD_logic_vector de 32 bit		Read_register for DMA INPUT.	
slv_reg_9	STD_logic_vector de 32 bit		OPEN pour un usage ultérieur.	
slv_reg_rden	STD_logic		Flag indiquant que l'adresse est prête à être lu. Le S_AXI_ARVALID du maitre vaut 1 Le axi_arready de l'esclave vaut 1 et axi_rvalid vaut 0 – Pas de donnée prête à être lu.	

slv_reg_wren	STD_logic	Flag indique que les données sont prête à être enregistré dans la mémoire. Slave : axi_wready = 1 Master : S_axi_wvalid = 1 Master : S_axi_awready = 1 Master : S_axi_awvalid = 1
reg_data_out	STD_logic_vector de 32 bit	Register to input or output data.
byte_index	int	Compteur permettant d'appliquer le STRB
aw_en	STD_logic	Flag permettant d'empêcher l'écriture d'une autre adresse dans que les données n'ont pas été envoyé Passe à 0 lorsque axi_awready = 1 passe à 1 (enregistrement d'une adresse) Passe à 1 uniquement après que S_AXI_bready et axi_bvalid sont à 1.

3) Validation

Test 3.1.1 : Validation Bloc PS/PL Test avec l'ILA.



Bloc design utilisé pour la validation.

L'ila est placé sur l'horloge générée par le quartz de la carte à 125MHz, cela est important car l'ILA requiert une horloge indépendante. Le reste du système est cadencé à 50Mhz à l'aide de l'horloge en sortie du processing system. Les entrées du bloc de commande initialement connectées aux sorties du DMA sont rendues externes et initialisées au niveau du wrapper.

Connecter ces entrées aux boutons de Debug ne fonctionne pas.

Le test est réussi si la modification des registres du bloc de commande PS-PL provoque les effets suivants :

Registre modifié	Adresse	Valeur 32bits	Valeur attendu	Réussi ?
Reg 0 - DMA-Start	0x43C00000	Initialisé à 0 passé à 1	1	
Reg 1 - DMA_IMG_W	0x43C00004	Initialisé à 0 passé à 0x00000280	1	
Reg 2 - DMA_IMG_H	0x43C00008	Initialisé à 0 passé à 0x000001E0	1E0	
Reg 3 - DMA_BA_LSB	0x43C0000C	Initialisé à 0 passé à 0x00400000	00400000	
Reg 4 - DMA_BA_MSB	0x43C00010	Initialisé à 0 passé à 0x11111111	11111111	
Reg 5 - VAL_SEUIL	0x43C00014	Initialisé à 0 passé à 0x000000C8	C8	
Reg 6 - SEL_Flux_1	0x43C00018	Initialisé à 0 passé à 0x11111111	1	
Reg 7 - SEL_Flux_2	0x43C0001C	Initialisé à 0 passé à 0x11111111	1	
Reg 8 - DMA_INPUT	0x43C00020	1 bouton est pressé puis le suivant, le registre est lu.		
Reg 7 - OPEN	0x43C00024	Initialisé à 0 passé à 0x11111111	1	

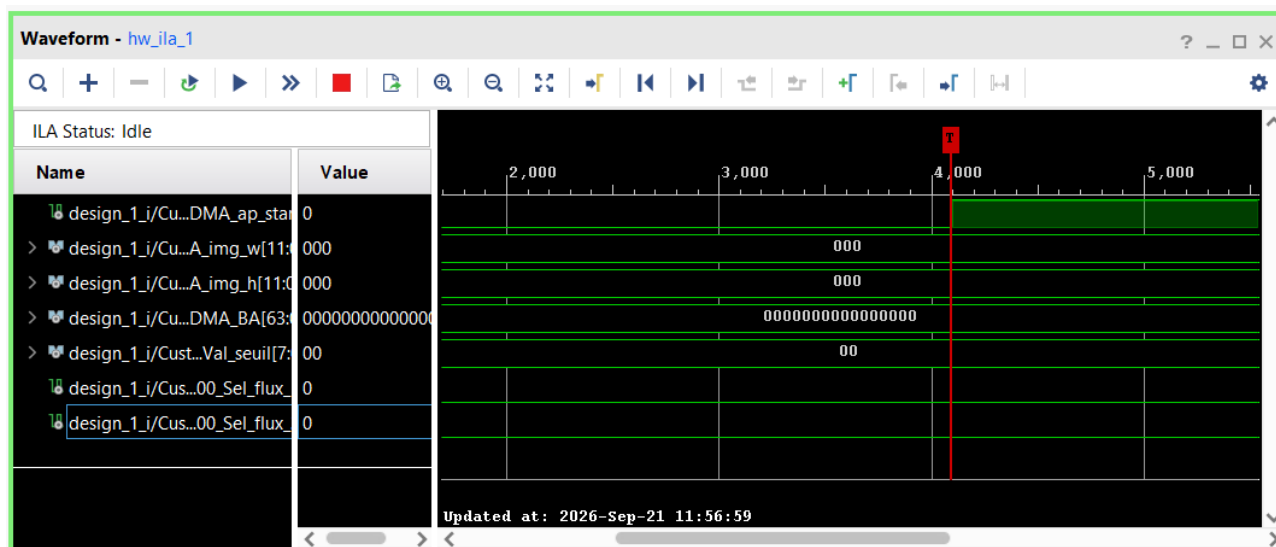
Read_all_value() est la fonction utilisée pour obtenir l'état de tout les registres, ils sont tous nul au début du test.

```

*****
Reading register for confirmation
*****
DMA_Start_val : 0
DMA_IMG_W_val : 0
DMA_IMG_H_val : 0
DMA_IMG_BA_LSB_val : 0
DMA_IMG_BA_MSB_val : 0
VAL_SEUIL : 0
SEL_Flux_1_val : 0
SEL_Flux_2_val : 0
DMA_INPUT_val : 0
*****

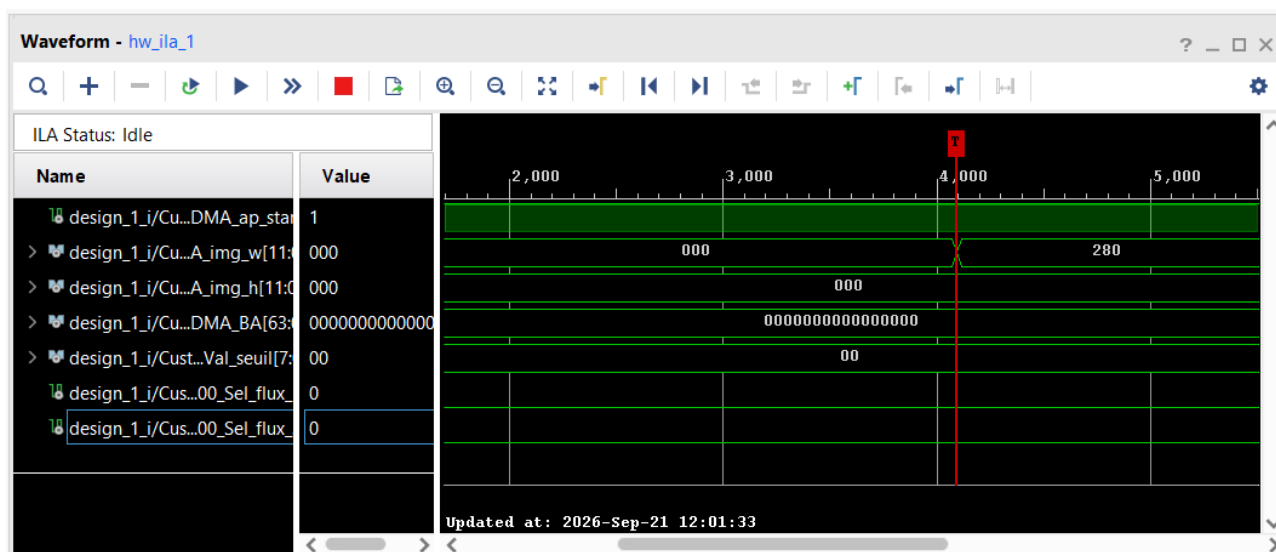
```

à l'initialisation, tout les registres valent 0x0000_0000

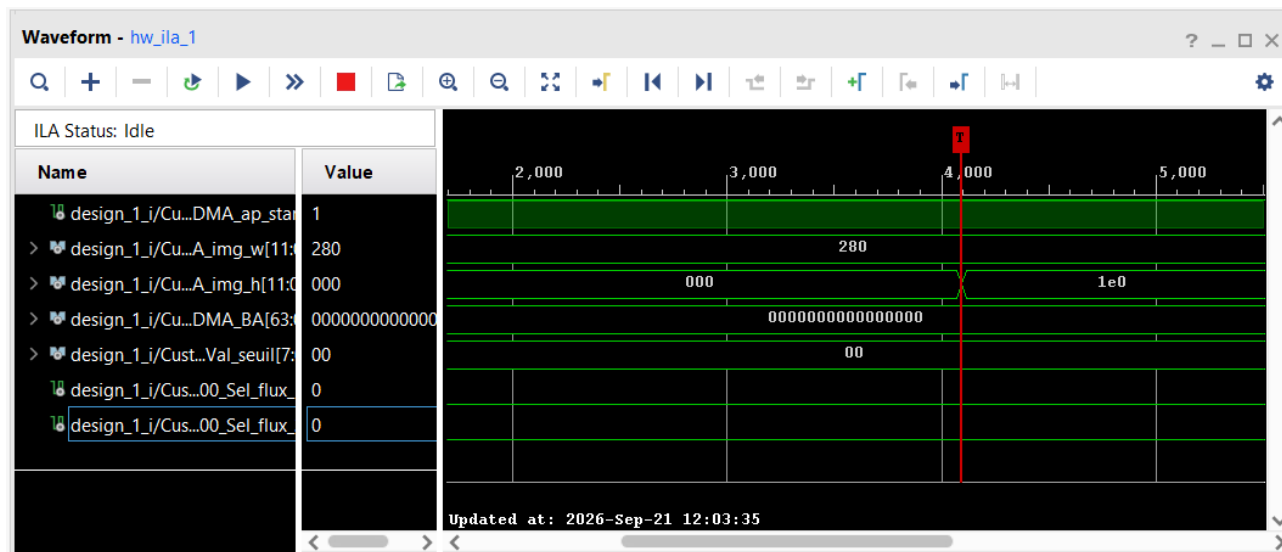


Le trigger de l'ILA est configuré pour un déclenchement sur front montant du signal DMA_AP_Start. À l'aide de Vitis en mode debug et des breakpoints, on modifie la valeur de chaque registre à l'aide de l'instruction Xil_out.

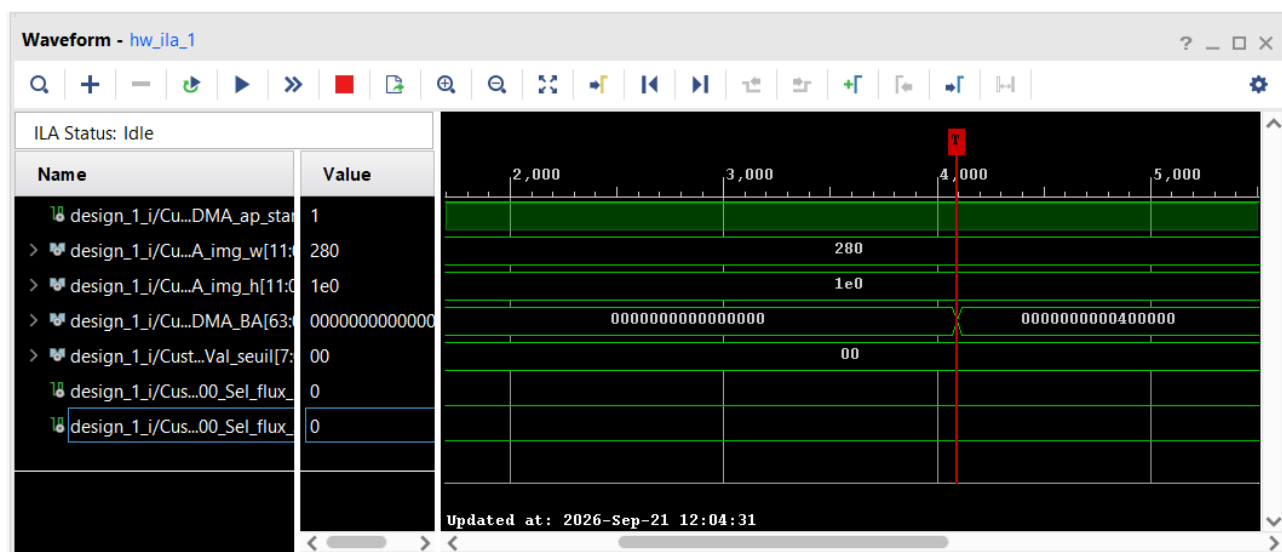
Après l'exécution de l'instruction Xil_out, le signal est détecté par l'ILA, la valeur de DMA_AP_START est bien passée à 1 après édition du registre.



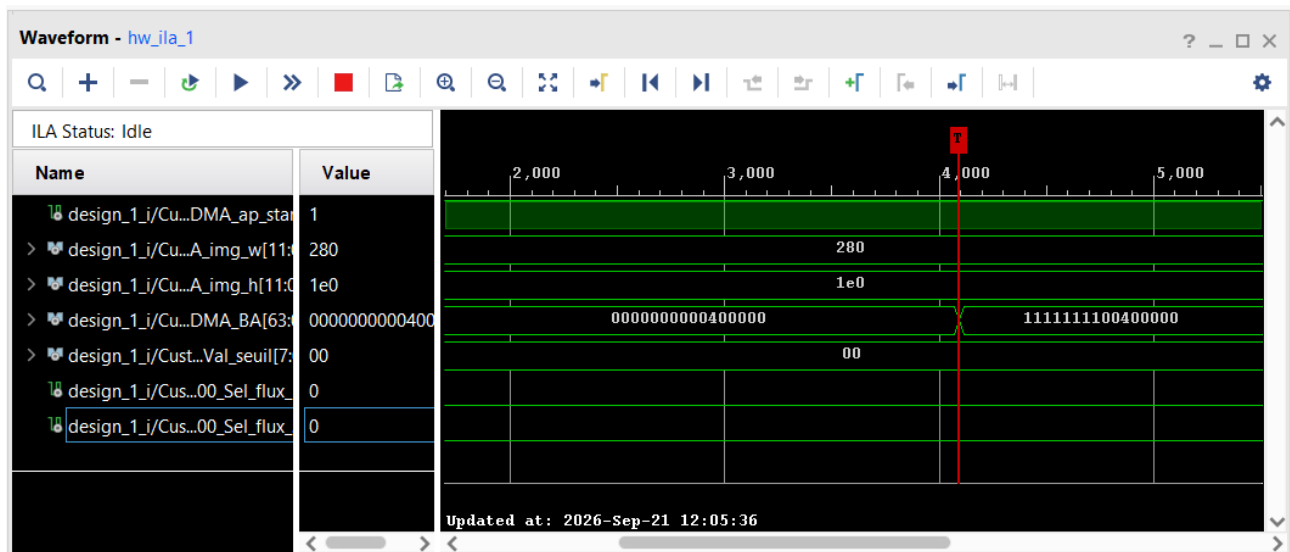
Le trigger de l'ILA est configuré pour détecter une valeur supérieure à 1 sur le signal DMA_IMG_W. L'édition du registre modifie bien la valeur de sortie.



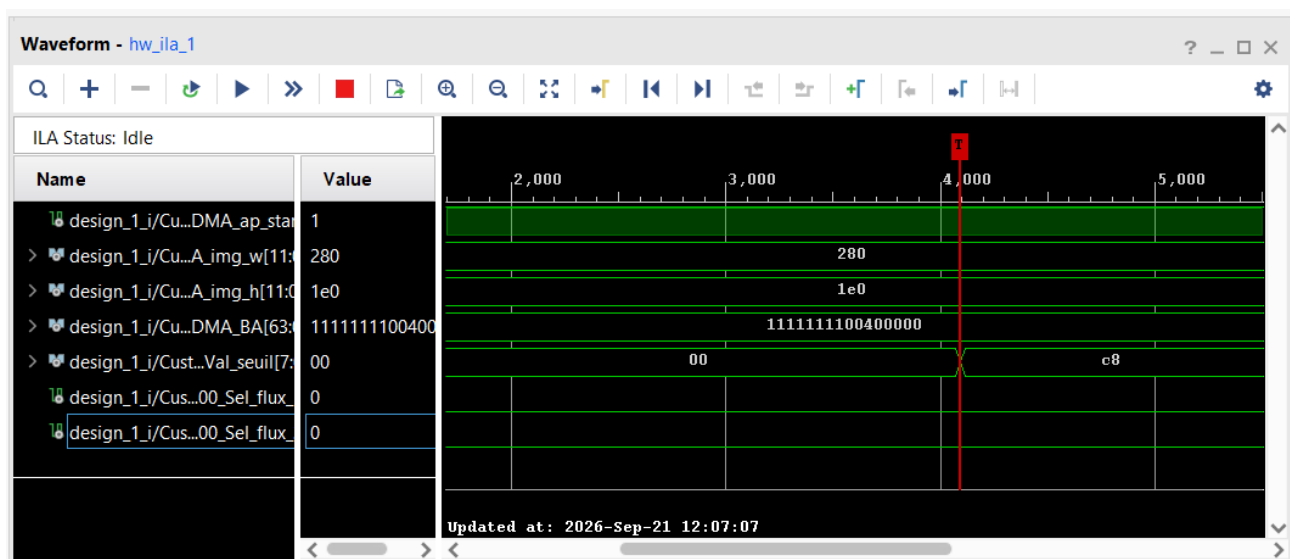
Le trigger de l'ILA est configuré pour détecter une valeur supérieure à 1 sur le signal DMA_IMG_h. L'édition du registre modifie bien la valeur de sortie.



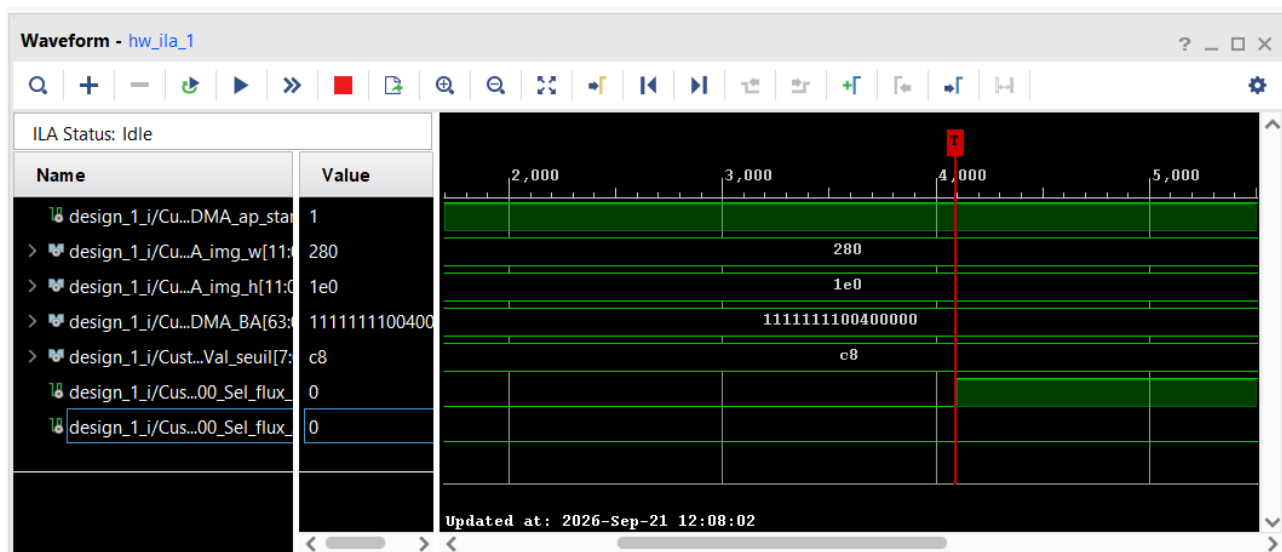
Le trigger de l'ILA est configuré pour détecter une valeur supérieure à 1 sur le signal DMA_IMG_BA. L'édition du registre modifie bien la valeur de sortie.



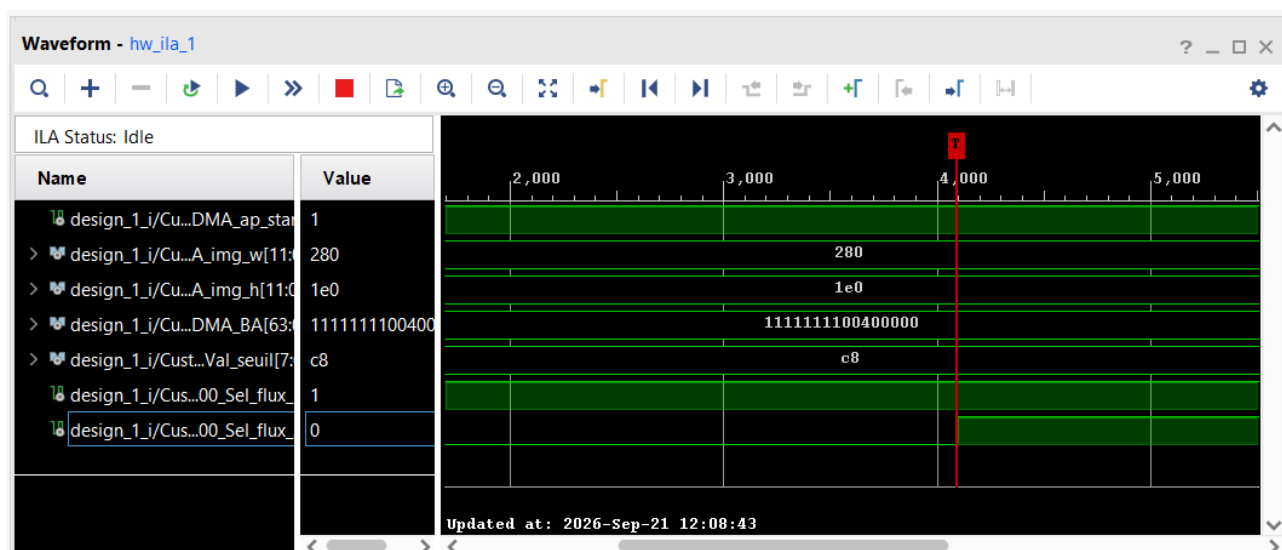
Le trigger de l'ILA est configuré pour détecter une valeur supérieure à 0x0000-0000-1000-0000 sur le signal DMA_IMG_BA. On observe une modification des 32 premiers bits de la variable.



Le trigger de l'ILA est configuré pour détecter une valeur supérieure à 1 sur le signal Val_seuil. L'édition du registre modifie bien la valeur de sortie.



Le trigger de l'ILA est configuré pour détecter une valeur supérieure à 1 sur le signal Sel_flux_1. L'édition du registre modifie bien la valeur de sortie.



Le trigger de l'ILA est configuré pour détecter une valeur supérieure à 1 sur le signal Sel_flux_2. L'édition du registre modifie bien la valeur de sortie.

Pour valider la prise en compte des entrées DMA, Celle-ci valant initialement '0' sont mis à 1 dans le wrapper.

Leur initialisation est indiquée par la valeur de DMA_INPUT_Val valant initialement 0 (cf screen putty au début du chapitre).

<pre> FIXED_IO_ps_srstb : inout STD_LOGIC; clk_125Mhz : in STD_LOGIC; s00_DMA_ap_done_0 : in STD_LOGIC := '1'; s00_DMA_ap_idle_0 : in STD_LOGIC:= '1' ; s00_DMA_ap_ready_0 : in STD_LOGIC := '1'); end design_1_wrapper; </pre>	<pre> ***** DMA_Start_val : 0 DMA_IMG_W_val : 0 DMA_IMG_H_val : 0 DMA_IMG_BA_LSB_val : 0 DMA_IMG_BA_MSB_val : 0 VAL_SEUIL : 0 SEL_Flux_1_val : 0 SEL_Flux_2_val : 0 DMA_INPUT_val : 7 DMA_OPEN : 0 ***** DMA_INPUT : 7 </pre>
--	--

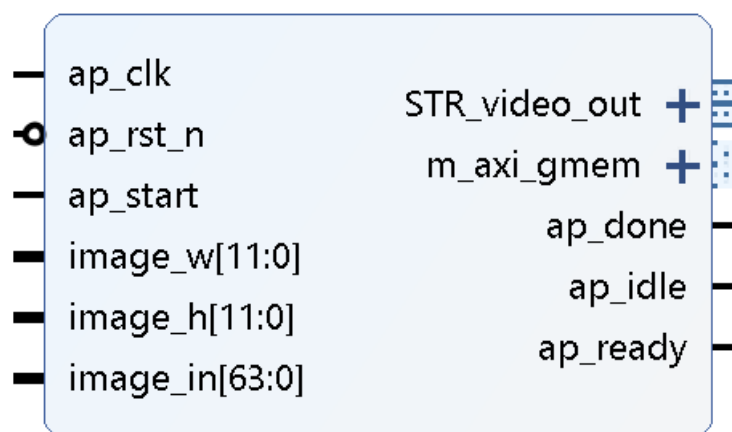
Modification de la valeur de ces variables dans le wrapper et affichage de la valeur du registre.

Placer ces inputs à 1 modifie bien la valeur du registre.

3.2 : Vérification du bon fonctionnement du DMA.

1/ Description fonctionnelle

Le DMA va directement discuter à l'aide d'un port axi4Full avec la mémoire DDR pour rendre disponible le flux de pixel sur le STR_video_out. (port Axi4 stream). Le bloc de contrôle mettant à disposition les informations récupérées sur les ports d'entrée du DMA sont un prérequis pour piloter ce bloc.



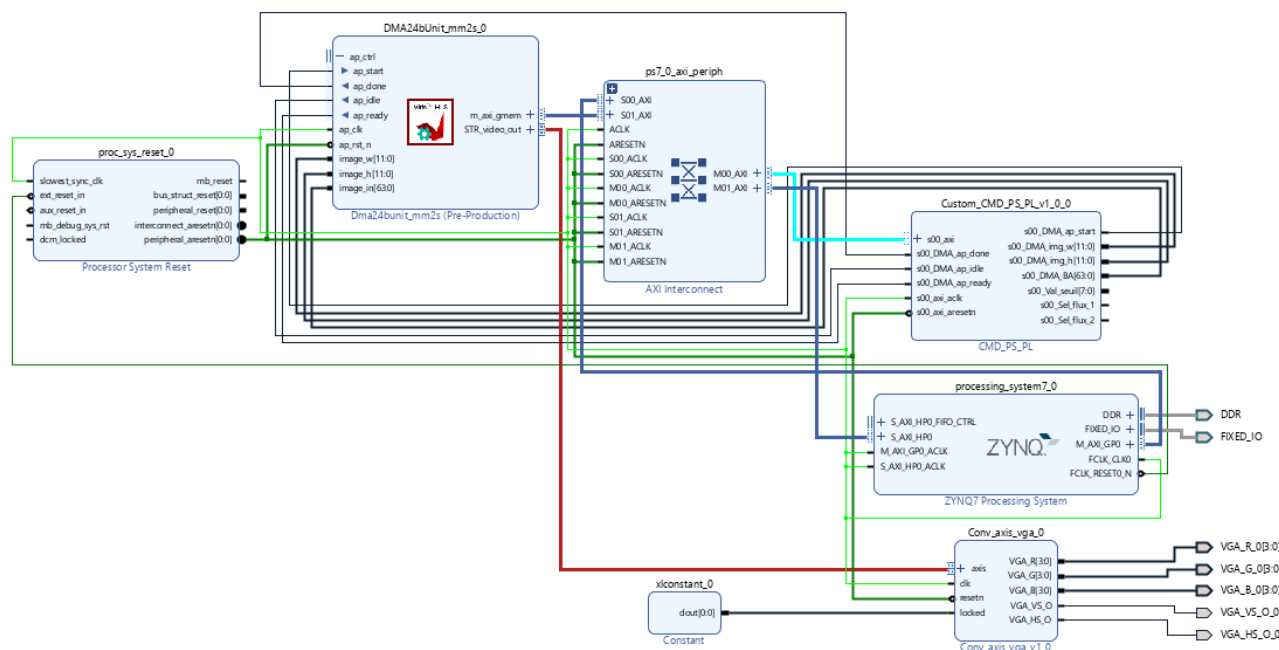
2/ Description des ports interne/externe

Description signaux externes			
Nom	Input/Output	Type	Description
ap_clk	INPUT	std_logic	Horloge
ap_rst_n	INPUT	std_logic	Reset not
ap_start	INPUT	std_logic	Enable, le signal est d'abord mis à 0, puis une fois que l'image est chargée dans la mémoire mis à 1 pour initier sa récupération
image_w	INPUT	Vecteur logique 12 bits	Width image
image_h	INPUT	Vecteur logique 12 bits	Height image
image_in	INPUT	Vecteur logique 64 bits	Base adresse register. (Premier Pixel de mon image)
STRvideo_out	OUTPUT	Axi4-STREAM	Output stream (64 bit?)
m_axi_gmem	OUTPUT	Axi4-FULL	Port de lecture vers la mémoire et les transmet sur le stream video.
ap_done	OUTPUT	std_logic	
ap_idle	OUTPUT	std_logic	
ap_ready	OUTPUT	std_logic	

3/ Test validation DMA

Le bloc DMA est fournit par Deverne, il sera validé d'une part par une analyse pour vérifier que le bloc est bien connecté puis associé au bloc de commande validé au chapitre précédent.

Test 2.2.1 : **Analyse** Validation à l'aide d'une analyse.



Nom du port	input/output	Commentaire
Ap_Start	INPUT - std_logic	Signal piloté par le bloc de commande.
Ap_clk	INPUT - std_logic	L'horloge est connectée à Fclk_clk0, il est impératif que l'horloge soit cadencée à 25Mhz, cette fréquence est modifiable dans le menu de configuration du ZYNQ. Des tests extensifs ont été réalisés avec une PLL pour générer une horloge à 25Mhz sans succès.
Ap_reseten	INPUT - std_logic	Le reseten est connecté au bloc processor system reset, ce composant est utilisé pour synchroniser les resets lorsque plusieurs horloges sont présentes.
image_W	INPUT - 32 bits	Signal piloté par le bloc de commande. Il est nécessaire de vérifier que la valeur du bus est bien initialisé à 480px.
image_H	INPUT – 32 bits	Signal piloté par le bloc de commande. Il est nécessaire de vérifier que la valeur du bus est bien initialisé à 640px.
image_in	INPUT – 64 bits	Signal piloté par le bloc de commande. Il est nécessaire de vérifier que la valeur du bus est initialisé à l'adresse du DDR en aménageant un espace pour le programme C.
ap_done	OUTPUT - std_logic	Signal récupéré par le bloc de commande. Ap_done et Ap_idle ont un comportement similaire, ces signaux remontent à la fin du chargement de l'image depuis la DDR.
ap_idle	OUTPUT - std_logic	Signal récupéré par le bloc de commande. Le signal doit être levée en attendant que DMA_START passe à 1.
ap_ready	OUTPUT - std_logic	Idem ap_done.
stream_video_out	OUTPUT - Axi4Stream	Stream directement connecté au bloc de pilotage de l'écran.
m_axi_gmem	OUTPUT - Axi4full	Axis4full connecté au DDR par l'intermédiaire de

		l'interconnect.
--	--	-----------------

Les signaux du DMA semblent être correctement connectés.

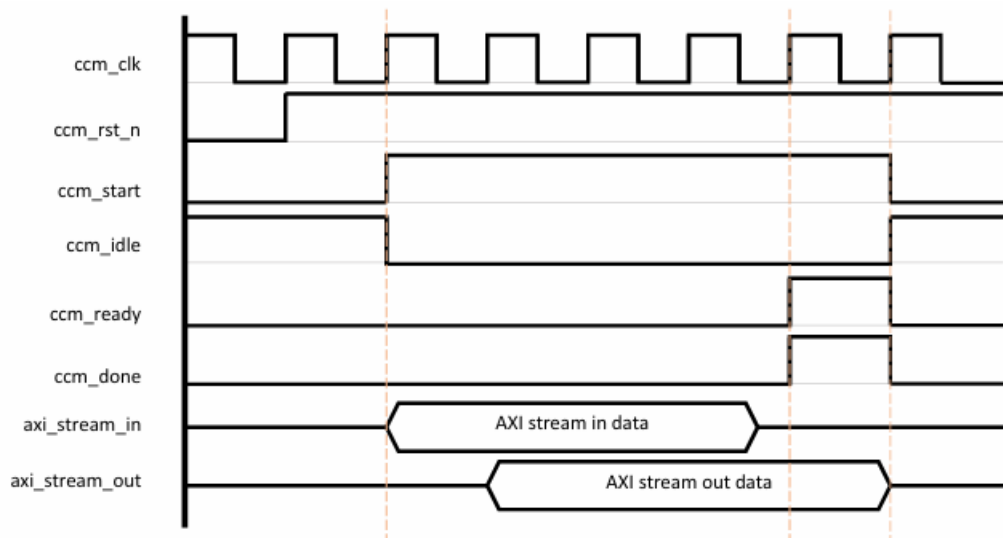


Figure 1: Control Interface chronogram

Chronogramme signaux d'Output DMA.

Comportement attendu des signaux de sortie du DMA.

Test 2.2.2 : **Test** à l'aide de l'ILA.

La procédure est la suivante :

- 1/ initialisation des registres du bloc de commande à l'aide de Vitis et du programme C.
- 2/ Chargement de l'image dans la mémoire DDR à l'aide du terminal XCST.
- 3/ Allumage DMA grâce à la modification du registre AP_Start dans le bloc de commande.

Le test est réussi si :

Signal	Comportement attendu	Réussi ?
Img W img H Img_in	Signaux initialisés à leur valeur par défaut par le bloc de commande.	
Ap_Idle	Idle est à l'état haut puis passe à l'état bas lorsque dma_start vaut 1. ce signaux remonte à la fin de la transmission Axis4full	
Ap_done et Ap_ready	Ces signaux remontent à la fin de la transmission Axis4full	
Liaison AXI4Stream	T_ready contrôlé par le bloc de pilotage de l'écran passe à 0 lorsque les compteurs sont dans les backporchs horizontaux et verticaux. T_valid vaut 1 indiquant que les données sont prêtes à être débitées.	

Liaison AXI4FULL	Il est attendu que le canal d'adresse dialogue avant que le canal de donnée ne s'active.	
	Affichage sur l'écran de l'image.	

1/ Initialisation des registres à l'aide de la fonction bloc_init qui utilise Xil_in(Add, Data)

```

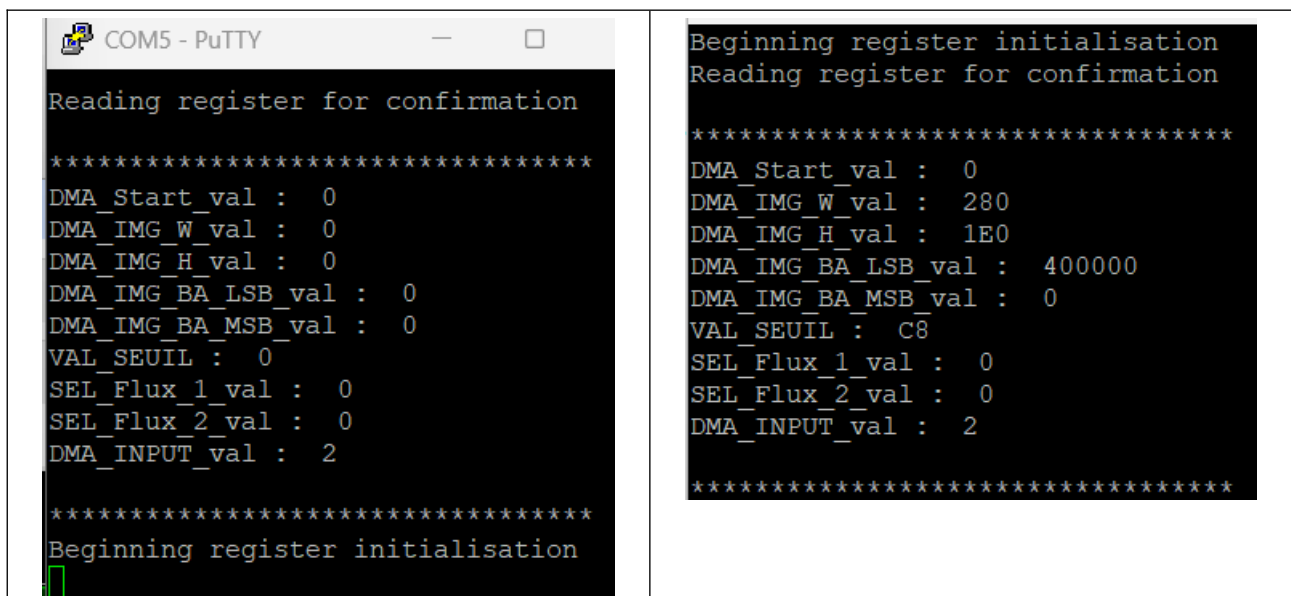
int main(){

    //xil_printf("erase img\n\r");
    //Generate_IMG(0);
    //xil_printf("load pattern\n\r");
    //Generate_IMG(1);
    xil_printf("Reading register for confirmation \n\r");
    read_all_value();
    xil_printf("Beginning register initialisation \n\r");
    bloc_init();
    //Change DMA read parameter to initiate the screen.
    //mwr -bin -file "C:/Users/cleme/Desktop/POEI/Cours/Projet_Corner_det
    Xil_Out32(AXI_DMA_Start_ADDRreg, 0x11111111);

    xil_printf("Reading register for confirmation \n\r");
    read_all_value();
    return 0;
}

```

Programme de test vitis.



Lecture des registres et initialisation.

DMA_input = 2 cela signifie que la sortie DMA_IDLE est à l'état haut.

Address	0 - 3	4 - 7	8 - B	C - F
44A00000	00000000	00000280	000001E0	00400000
44A00010	00000000	000000C8	00000000	00000000
44A00020	00000002	00000000	00000000	00000000

Vérification dans la mémoire que les registres ont bien été initialisé.

Les registres sont correctement initialisées. On constate que l'outil de debug indique que d'autres registres sont initialisés de manière cyclique. *Ce serait un Artefact.*

3/ Chargement de la mémoire DDR à l'aide de la console XSCT et de la commande suivantes :

```
mwr -bin -file "C:/Users/cleme/Desktop/POEI/Cours/Projet_Corner_detect/Vitis_WD/PS_prog/sample_in_rgb.bin" 0x00400000 307200
```

Vérification que la mémoire à bien été initialisée :

0x00400000 : 0x400000 <Hex Integer>  New Renderings...					0x00400000 : 0x400000 <Hex Integer>  New Renderings...				
Address	0 - 3	4 - 7	8 - B	C - F	Address	0 - 3	4 - 7	8 - B	C - F
00400000	00000000	00000000	00000000	00000000	00400000	D6F9E3D7	E2D6F8E2	F7E1D5F8	D6F8E2D6
00400010	00000000	00000000	00000000	00000000	00400010	E2D6F8E2	F7E1D5F8	D7F7E1D5	E2D7F8E2
00400020	00000000	00000000	00000000	00000000	00400020	F9E3D8F8	D8F8E2D7	E3D8F9E3	FAE4D8F9
00400030	00000000	00000000	00000000	00000000	00400030	D8FAE4D8	E3D8FAE4	F9E3D8F9	DAF9E3D9
00400040	00000000	00000000	00000000	00000000	00400040	E2DAF9E2	F9E2DAF9	DAF9E2DA	E2DAF9E2
00400050	00000000	00000000	00000000	00000000	00400050	F9E2DAF9	DAF9E2DA	E2DAF9E2	F9E2D9F9
00400060	00000000	00000000	00000000	00000000	00400060	D8F9E3D8	E3D8F9E3	F9E3D8F9	D8F9E3D8
00400070	00000000	00000000	00000000	00000000	00400070	E3D8F9E3	F9E3D8F9	D7F8E2D7	E3D8F8E2
00400080	00000000	00000000	00000000	00000000	00400080	F9E3D8F9	D8F9E3D8	E3D8F9E3	F9E3D8F9
00400090	00000000	00000000	00000000	00000000	00400090	DBF9E3D9	E3D8FAE3	F9E3D8FA	DCF9E4DC
004000A0	00000000	00000000	00000000	00000000	004000A0	E4DDF9E4	FAE4DEFA	DDFAE4DE	E5DDFAE5
004000B0	00000000	00000000	00000000	00000000	004000B0	FAE5DDFA	DDFAE5DD	E5DDFAE5	FAE4DEFA
004000C0	00000000	00000000	00000000	00000000	004000C0	DDF9E3DD	E4DCF9E4	F9E4DCF9	DCF9E4DC
004000D0	00000000	00000000	00000000	00000000	004000D0	E4DCF9E4	F9E4DCF9	DCF9E4DC	E4DCF9E4
004000E0	00000000	00000000	00000000	00000000	004000E0	FAE5DDF9	DDFAE5DD	E5DDFAE5	FAE5DDFA

4/ La valeur dans le registre DMA_start est modifiée pour initier la lecture de la mémoire DDR par le DMA.

```
xil_Out32(AXI_DMA_Start_ADDReg, 0x11111111);
```

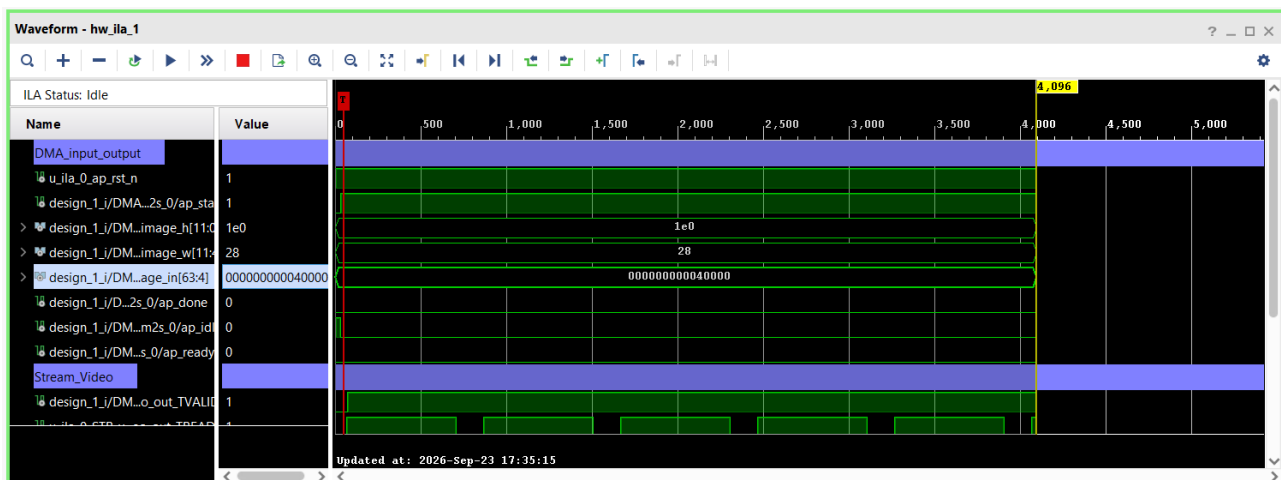
5/ L'image suivante apparaît sur l'écran



On observe des zones de pixels identiques car on a réduit la quantité de nuance de couleurs. Il y a aussi un problème de synchronisation entre le bloc de pilotage de l'écran et le DMA.

Test 3.2.2 : **Test** à l'aide de l'ILA.

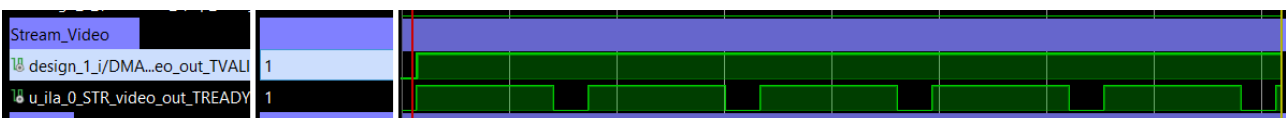
Trigger sur $ARVALID = 1 \text{ AND } ARREADY = 1$ afin de monitorer la liaison axifull.



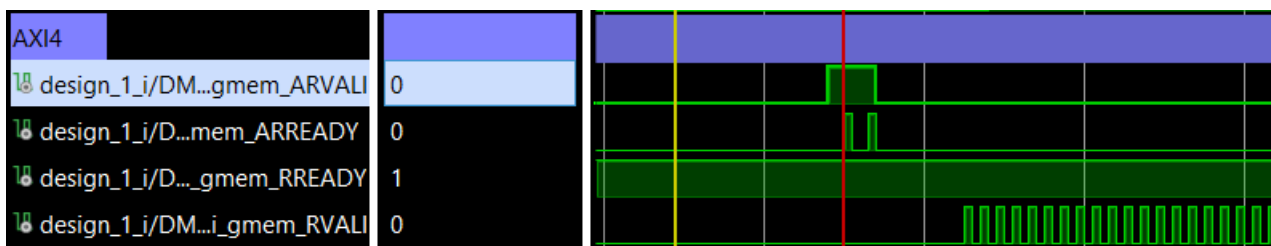
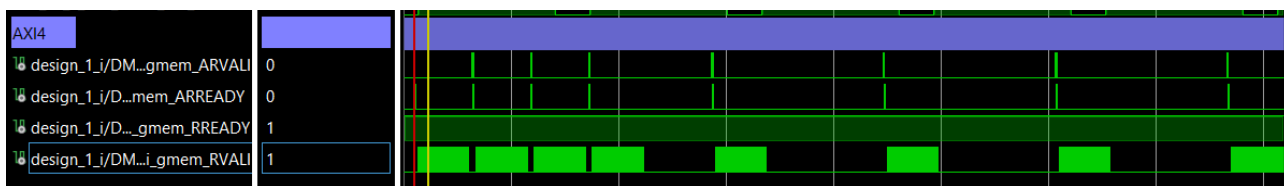
La Valeur est entrées (img_in, img_W, img_H, ap_rstn) sont conforme au résultat attendu.

On observe que Ap_idle descend lorsque ap_start monte.

Done, idle et ready restent à l'état bas en attendant la fin de la transmission.



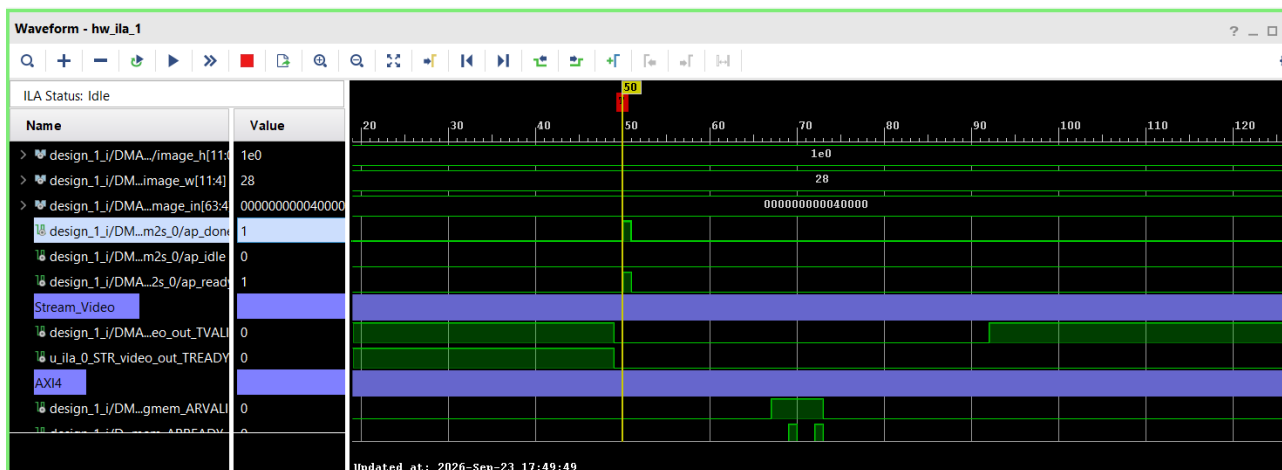
Au niveau du bus de transmission video, on constate que Tvalid reste à l'état haut car le DMA est toujours prêt à émettre. Le Tready commandé par Conv axis-VGA passe à l'état bas lorsqu'on est hors image utile. (Back porch des colonnes)



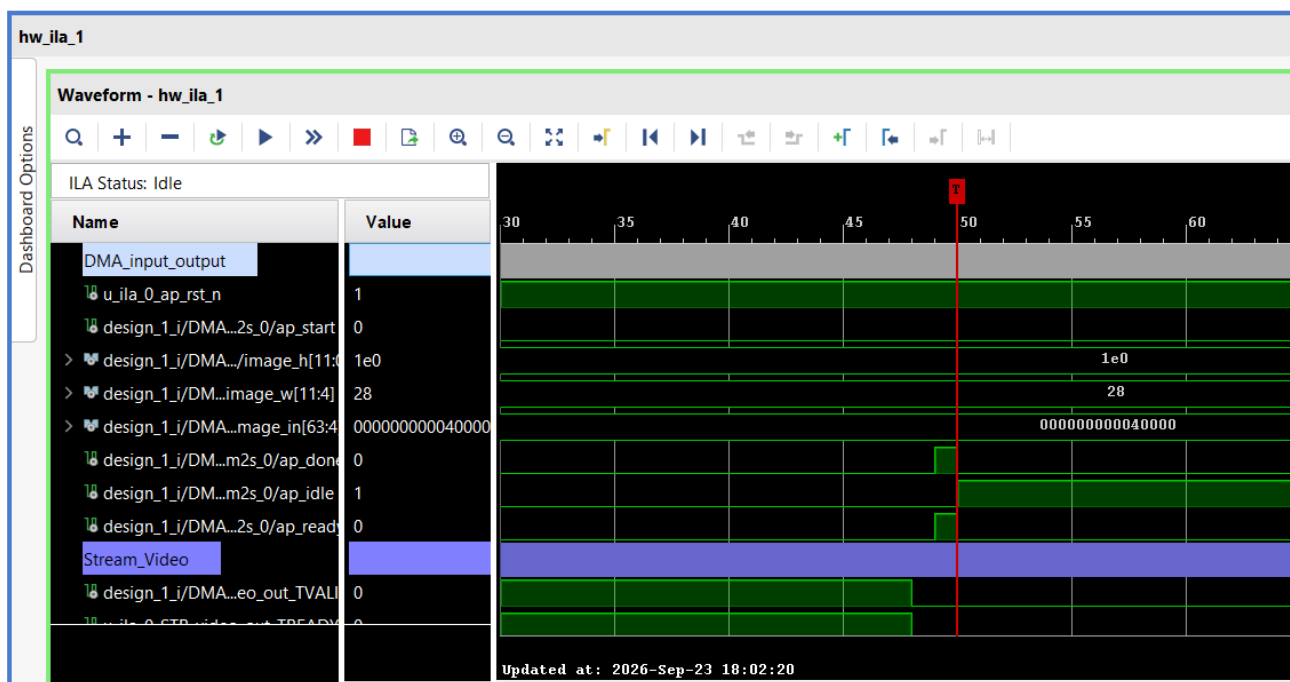
Au niveau de l'axis, on constate que ARValid arrive en premier (adresse de lecture mis à disposition par le DMA), puis ARReady vaut 1, récupération de l'adresse par le DMA)

Puis le ARVALID descend et la transmission des données peut commencer entre le DMA et la DDR. Côté DDR Rready vaut toujours 1.

La lecture par le DDR d'un autre flux de données n'est déclenchée qu'après la mise à l'état haut de ARVALID.



En fin de transmission Ap_done et ap_idle passent à 1 puis repassent à 0 car ap_start est à 1.



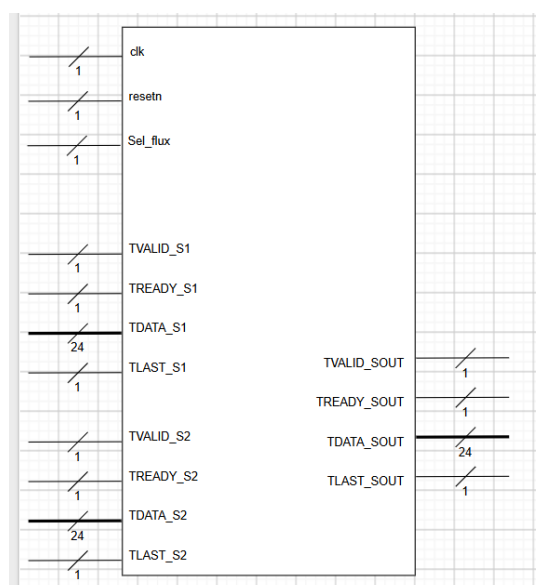
Lorsque Start = 0, à la fin de la transmission. Idle reste à 1.

3.X Chaîne de traitement

3.1 Bloc de Sélecteur de flux

1/ Description fonctionnelle

Le sélecteur de flux intervient à deux reprises dans le synoptique, représenté par un multiplexeur il est piloté par le bloc de pilotage. La valeur de l'entrée sel_flux permet de choisir un flux ou un autre. Une version de ce bloc sélecteur aura le flux de sortie dupliquée.



Le registre de commande peut être initialisé au démarrage ou modifié en fonctionnement, il faut alors attendre une fin d'image que l'on contrôlera à l'aide de compteur (à implémenter) pour basculer sauf après un reset où le flux doit être redirigé immédiatement.

Le design sera testé dans un premier temps avec un test bench pour étudier les signaux puis ajouté au design général.

2/ Description des signaux internes et externes.

Signaux Externes			
Nom du signal	Input/ Output	Type de signal	Commentaire
Clk	INPUT	Std_logic	Horloge de cadencement du système
resetn	INPUT	Std_logic	Resetn permettant l'initialisation des registres internes.
sel_flux	INPUT	Std_logic	Lorsque Sel_flux vaut 0 c'est un flux qui est choisi, lorsqu'il vaut 1 c'est l'autre.
Stream out	Output	Axis-stream	Flux de sortie
Stream 1	INPUT	Axis-stream	Premier flux d'entrée
Stream 2	INPUT	Axis-stream	Second flux d'entrée
Signaux Internes			
Nom du signal		Type de signal	Commentaire
S_sel_flux_reg		std_logic	Registre de la sélection du flux
S_flag_reset		std_logic	Signal permettant de prendre en compte qu'un reset vient d'être effectué et qu'il n'est nécessaire d'attendre un Tlast pour appliquer le choix du flux.
S_flag_sel_flux		std_logic	Signal pilotant directement le stream de sortie. Il permet de prendre en compte le changement à chaud de flux.
S_axis_tdata		std_logic_vector	Registre du bus de données de la liaison Axis
S_axis_tlast		std_logic	Registre de Tlast du stream Axis
S_axis_tready		std_logic	Registre de Tready du stream Axis
S_axis_tvalid		std_logic	Registre de Tvalid du stream Axis

3/ Test et validation du bloc.

Test 2.3.1 Testbench de validation du bloc.

Signal	Comportement attendu	Validé ?
Tdata_out	Tdata_out est égal à Tdata_1 si sel flux = 0, Tdata_2 si sel flux = 1.	
Tlast_out	Tlast_out est égal à Tlast_1 si sel flux = 0, Tlast_2 si sel flux = 1.	
Tready_in	Tready_in est égal à Tready_out_1 si sel flux = 0, Tready_out_2 si sel flux = 1.	
Tvalid_out	Tvalid_out est égal à Tvalid_1 si sel flux = 0, Tvalid_2 si sel flux = 1.	



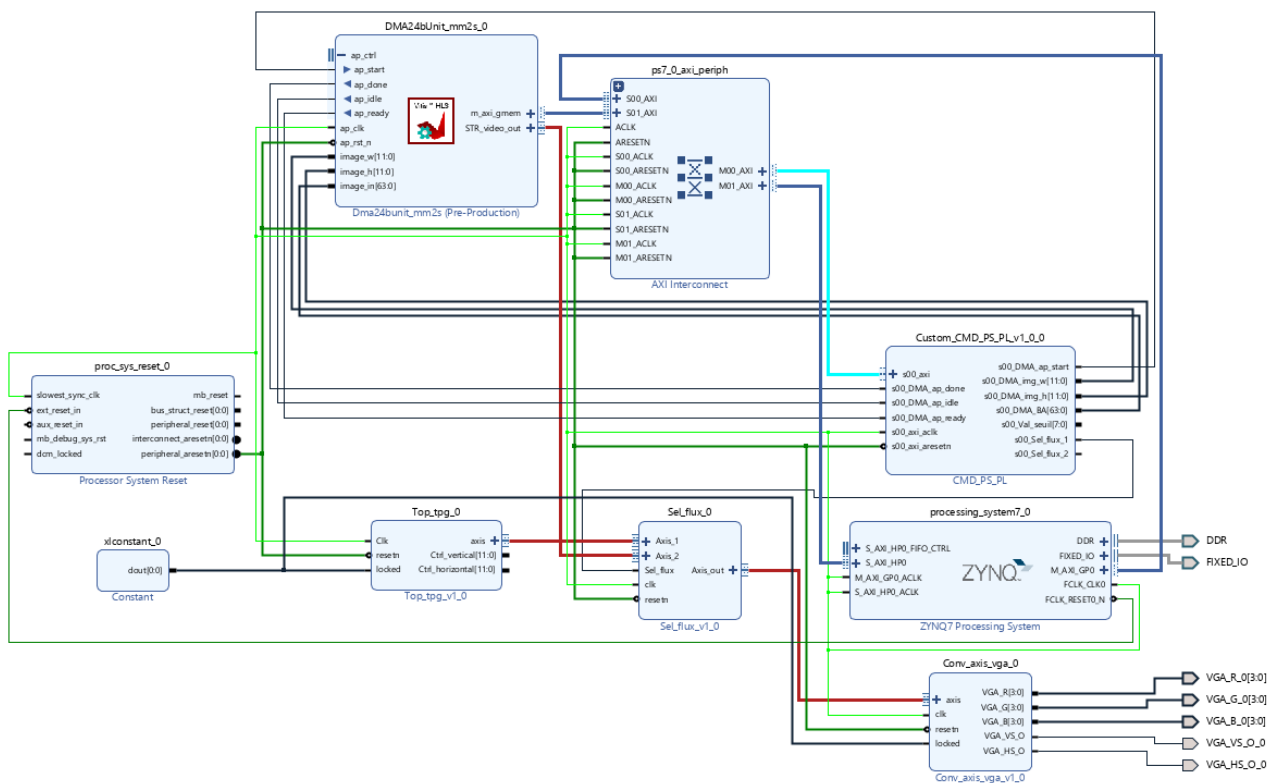
Test bench bloc de sélecteur de flux.

Le signal S_Sel_flux est l'image du registre de sélection du flux, lorsqu'il vaut 0, le flux_1 est retransmis sur la sortie, lorsqu'il vaut 1 c'est le flux 2.

Le S_flag_sel_flux est le signal qui pilote directement la logique combinatoire de sélection du flux.

Après un reset, il faut qu'un flux soit directement redirigé sans attendre de condition sur le Tlast. S_axis_data_out applique directement la valeur du flux_1 c'est le comportement attendu.

Par la suite, le changement de flux intervient après un changement du signal S_sel_flux et à la fin d'une transmission (lorsque Tlast vaut 1). C'est ce qui est démontré au niveau du marqueur jaune.

3.1.2 : **Démonstration** du changement de flux vidéo.

Le bloc de sélection de flux est connecté au TPG, à la sortie vidéo du DMA ainsi qu'à la sortie Sel_flux du bloc de commande. Le programme suivant va être utilisé pour tester le projet :

```

int main(){

    xil_printf("Lecture des registres\n\r");
    read_all_value();
    xil_printf("Initialisation des registres\n\r");
    bloc_init();
    //Chargement de l'image dans la mémoire.
    //mwr -bin -file "C:/Users/cleme/Desktop/POEI/Co
    //Démarriage DMA.
    Xil_Out32(AXI_DMA_Start_ADDRreg, 0x11111111);

    xil_printf("Lecture des registres\n\r");
    read_all_value();
    //Changement à chaud du flux de lecture.
    Xil_Out32(AXI_SEL_Flux_1_ADDRreg, 0x11111111);
    xil_printf("Lecture des registres\n\r");
    read_all_value();
    Xil_Out32(AXI_SEL_Flux_1_ADDRreg, 0x00000000);

    return 0;
}

```

```

Lecture des registres

*****
DMA_Start_val : 0
DMA_IMG_W_val : 0
DMA_IMG_H_val : 0
DMA_IMG_BA_LSB_val : 0
DMA_IMG_BA_MSB_val : 0
VAL_SEUIL : 0
SEL_Flux_1_val : 0
SEL_Flux_2_val : 0
DMA_INPUT_val : 2

*****
Initialisation des registres
Lecture des registres

*****
DMA_Start_val : 11111111
DMA_IMG_W_val : 280
DMA_IMG_H_val : 1E0
DMA_IMG_BA_LSB_val : 400000
DMA_IMG_BA_MSB_val : 0
VAL_SEUIL : C8
SEL_Flux_1_val : 0
SEL_Flux_2_val : 0
DMA_INPUT_val : 0

*****
Lecture des registres

*****
DMA_Start_val : 11111111
DMA_IMG_W_val : 280
DMA_IMG_H_val : 1E0
DMA_IMG_BA_LSB_val : 400000
DMA_IMG_BA_MSB_val : 0
VAL_SEUIL : C8
SEL_Flux_1_val : 11111111
SEL_Flux_2_val : 0
DMA_INPUT_val : 0

*****

```

Cf vidéo jointe : test_3_2_Demo_sel_flux.

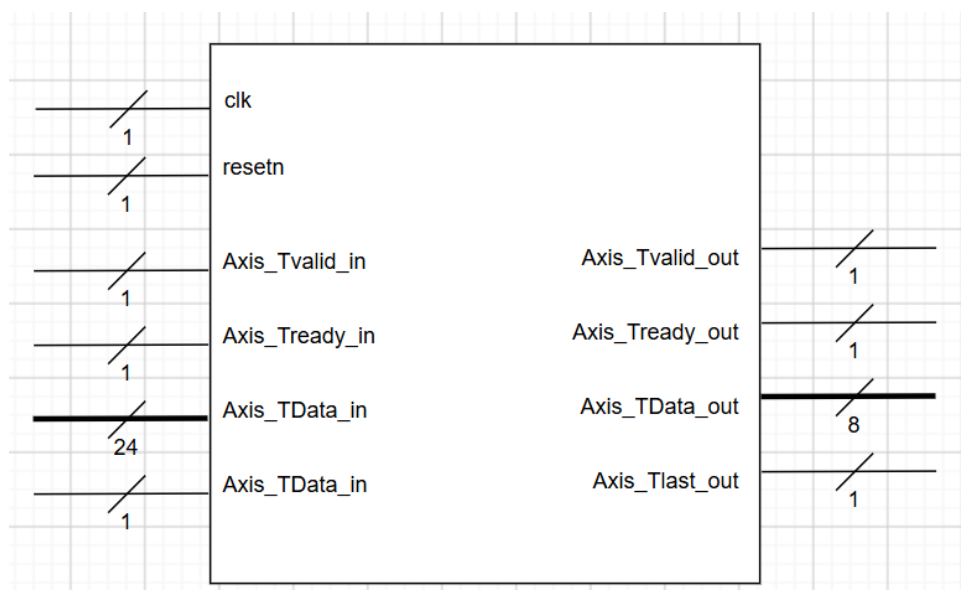
A l'initialisation, la valeur du registre SEL_FLUX_1 est égal à 0, on obtient sur l'écran le TPG. L'image apparaît au changement du registre après avoir chargé l'image dans la DDR et démarré le DMA. On constate qu'il y a toujours cette désynchronisation.

Après conditionnement de l'incrémentation des signaux à Tready_out = 1, la désynchronisation au démarrage a été corrigée. La prise en compte d'un changement de flux en fonctionnement n'est pas encore implémentée.

3.2 Bloc de conversion luminance

1/ Description Fonctionnelle

Ce bloc est le premier de la chaîne de traitement, il réalise une opération de moyennage entre les trois canaux RGB afin d'obtenir une image en nuance de gris. L'entrée est en Axi4Stream avec Tdata valant 24bit (un octet par canaux RGB). La sortie est également en Axi4Stream en 8 bit car la luminance est ramenée entre 0 et 255.



$$V_{gris} = Arrondi\left(\frac{V_{rouge} + V_{vert} + V_{bleu}}{3}\right)$$

2/ Description des signaux internes et externes

Signaux Externes			
Nom du signal	Input/ Output	Type de signaux	Commentaires
C_Axis_Tdata_out_width	Generic		Permet de choisir la taille du vecteur de sortie (24 ou 8).
Clk	Input	Std_logic	Horloge du signal
Resetrn	Input	Std_logic	Reset not du bloc
Axis_Tvalid_in	Input	Std_logic	Signal indiquant que les données sont prêtes à être traitées.
Axis_Tready_in	Output	Std_logic	Le bloc de conversion utilise un process combinatoire pour le calcul qui n'a pas besoin de

			stocker des données. La condition pour que le bloc de traitement soit prêt à recevoir des données est que le bloc suivant soit prêt.
Axist_TData_in	Input	Std_logic_vector	Vecteur 24 bits RGB.
Axist_TLast_in	Input	Std_logic	Ce signal est répété en sortie en prenant soin de ne pas désynchroniser les données.
Axis_Tvalid_out	Output	Std_logic	Signal indiquant que les données sont prêtes à être débitées. C'est l'image du Tvalid du bloc en amont de la chaîne de traitement.
Axis_Tready_out	Input	Std_logic	Signal permettant au bloc en aval de la chaîne de traitement d'indiquer qu'il est prêt à recevoir des données.
Axist_TData_out	Output	Std_logic_vector	Vecteur 8 bits contenant la valeur des pixels RGB moyennée.
Axist_TLast_out	Output	Std_logic	Image de TLast_out
Signaux Externes			
Nom du signal	Type de signaux	Commentaires	
S_axis_Tlast_reg	Std_logic	Registre d'enregistrement du signal Tlast.	
S_axis_Tready_reg	Std_logic	Registre d'enregistrement du signal Tready	
S_Axis_TValid_reg	Std_logic	Registre d'enregistrement du signal Tvalid	
S_Axis_Tdata_reg	Std_logic_vector (24 bits)	Registre d'enregistrement du vecteur Tdata	
S_grayvalue	Std_logic_vector (8 bits)	Valeur moyennée calculé à l'aide de l'intensité lumineuse RGB	

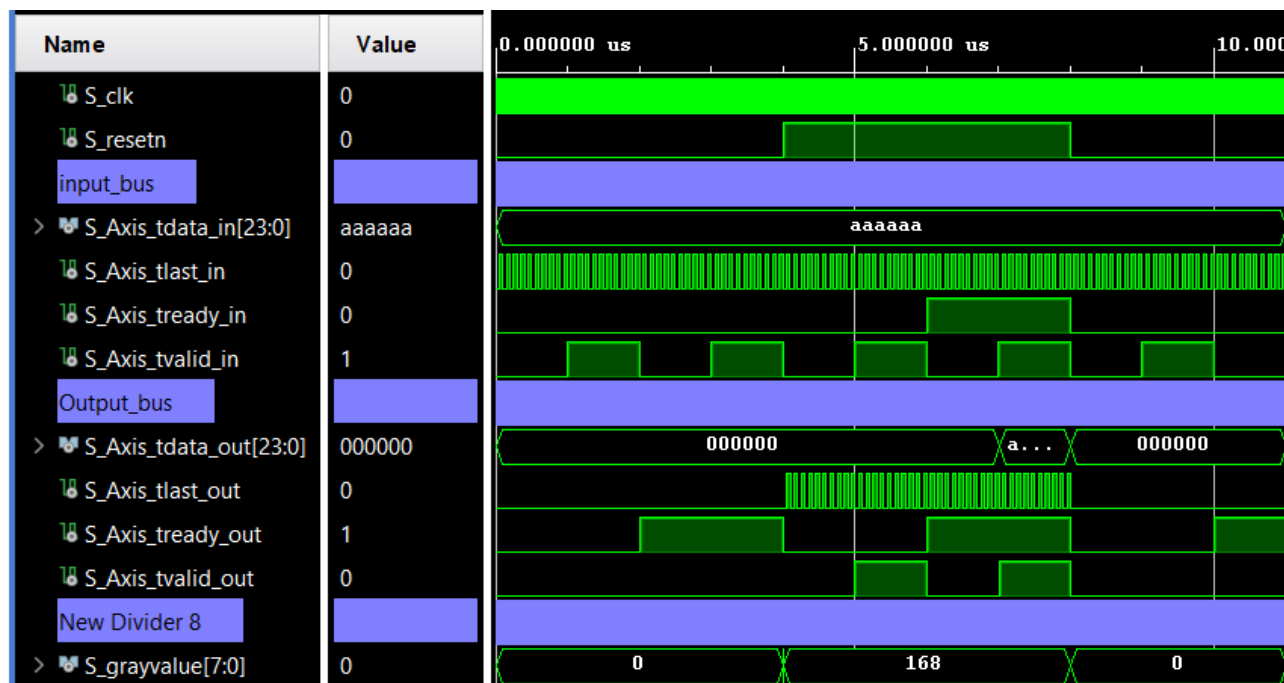
3/ Test et validation

3.2.1 Validation à l'aide d'une **simulation**.

Le test en simulation est réalisé en appliquant un vecteur data connu, sa valeur est en hexadécimal « aaaaaa ». On applique un signal carré à l'entrée de Tlast, En ce qui concerne Axis_Tready_out et Axis_Tvalid_in, on les fait varier de façon à couvrir toutes les possibilités.

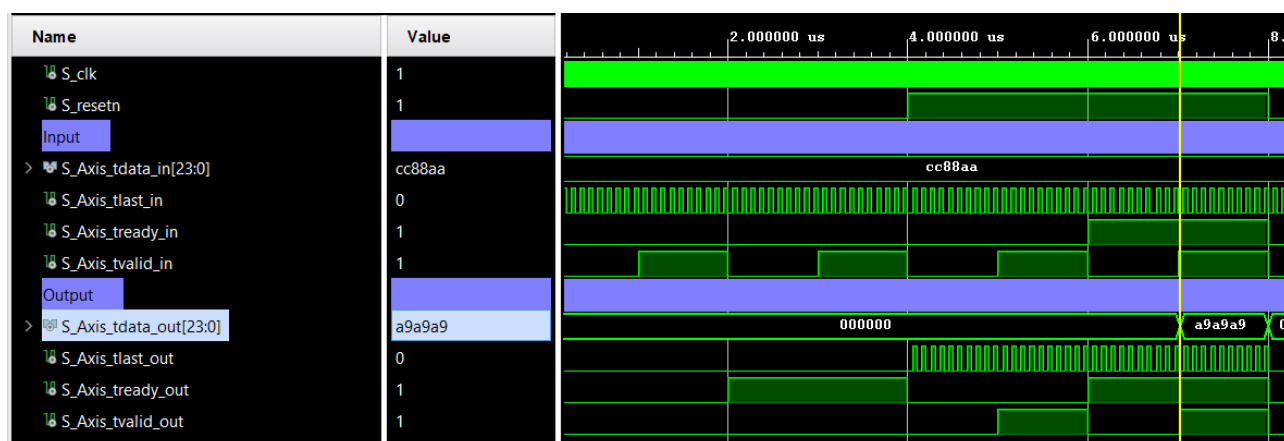
Signal	Comportement attendu	Réussi ?
Axis_Tready_in	Tready_in vaut 0 quand reset est actif, le signal est autrement décalé d'un coup d'horloge par rapport à Axis_Tready_out.	
Axis_Tvalid_out	Idem pour Axis_Tvalid_out.	
Axist_TData_out	La valeur en sortie de Tdata_out est égale à la moyenne des valeurs	

	RGB. La valeur est actualisé seulement si Tvalid_out = 1 et Tvalid_in = 1	
Axist_TLast_out	La valeur de T_last_out est égale à T_last_in décalé d'un coup d'horloge. T_last vaut 0 sur reset actif.	



Lorsque S_resetrn est actif, Tready_in et tvalid_out valent toujours 0, Les données sur S_axis_tdata_out sont égal à 0 sur resetrn actif et les données ne sont mis à disposition que lorsque T_valid_in et S_axis_tready_out sont actif.

S_axis_tlast_out est transmis même lorsque tready_out est à l'état bas, ce qui n'arrivera jamais en situation réelle car si T_ready_out est inactif le bloc n'émettra pas ce signal.

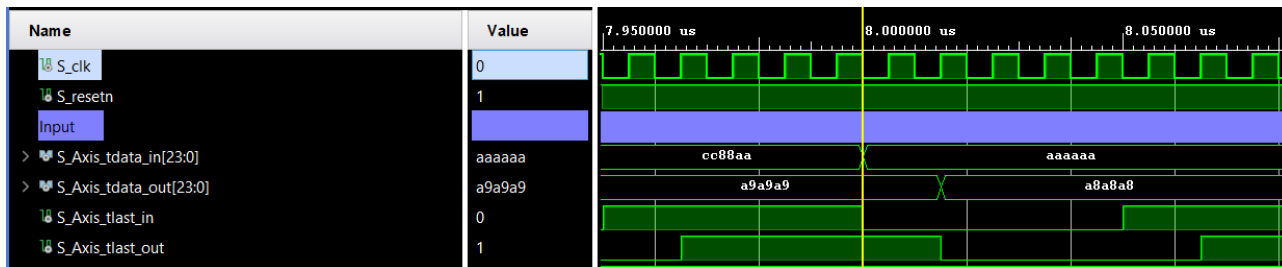


On applique en entrée du bloc de moyennage la valeur « CC88AA », La valeur du canal rouge vaut 0xAA, celle du vert vaut 0x88, enfin le bleu vaut 0xCC. Calculons la moyenne :

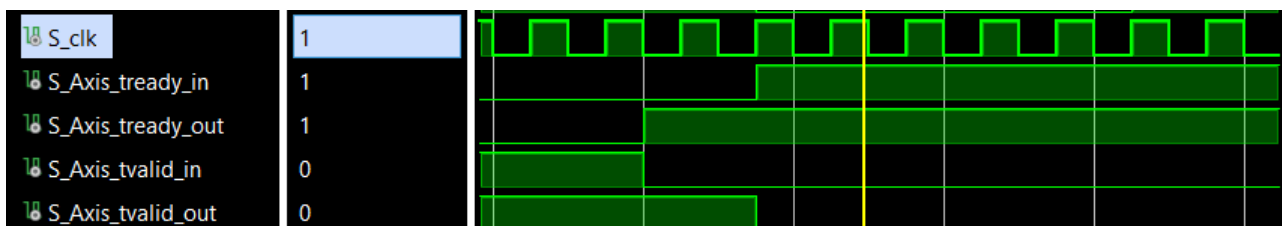
$$\frac{(204+136+170)}{3} = 170$$

Soit 0xAA en hexa, 0xA9 = 169 en décimal, la conversion est correcte à l'arrondi près car pour être synthétisable, le calcul est implémenté en divisant chaque valeur par 3.

Il faut prendre garde à ce que les signaux subissent un retard équivalent, dans notre cas l'opération de calcul de la valeur de gris prend deux coups d'horloge, il faut que le même retard soit égal sur tout les signaux.



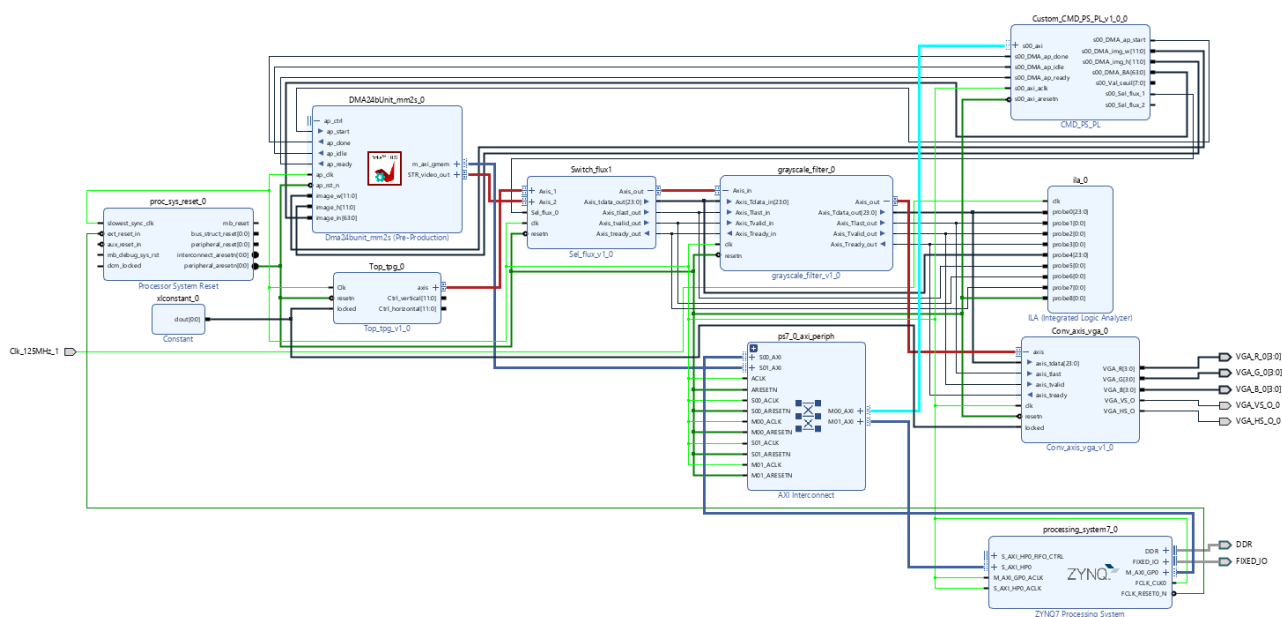
Mis en évidence du temps de rétention Des signaux last_in et last_out dans le bloc.



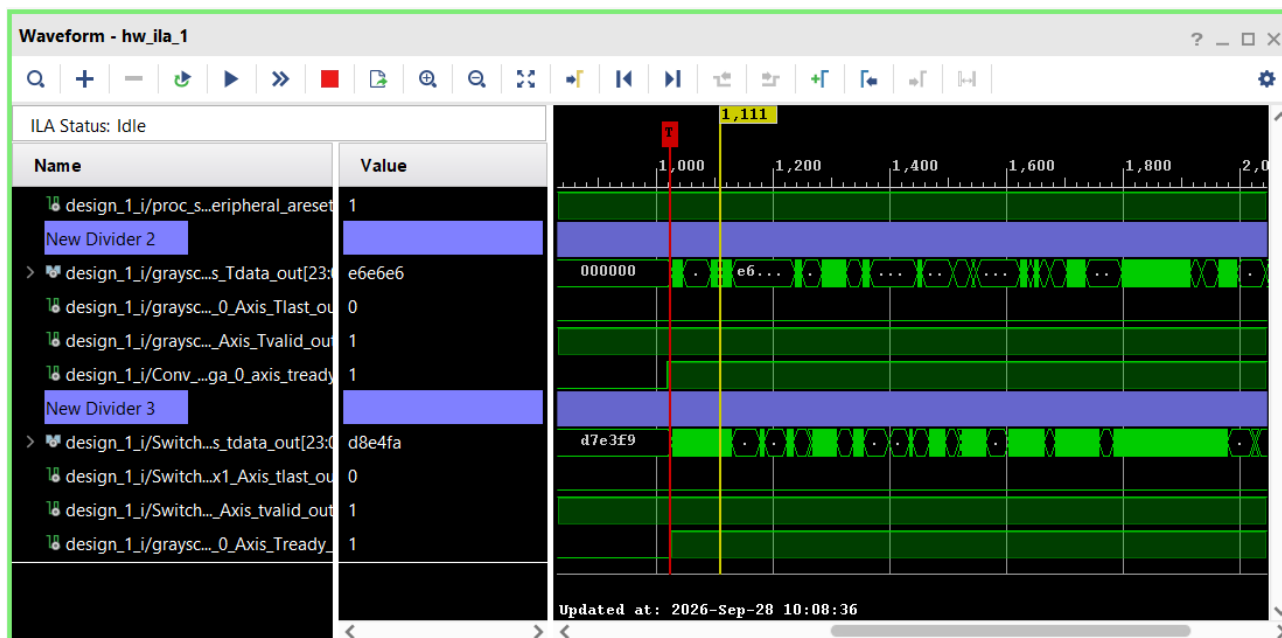
Mis en évidence du temps de rétention Des signaux Tready et Tvalid dans le bloc.

Le temps de rétention de ces signaux est égal.

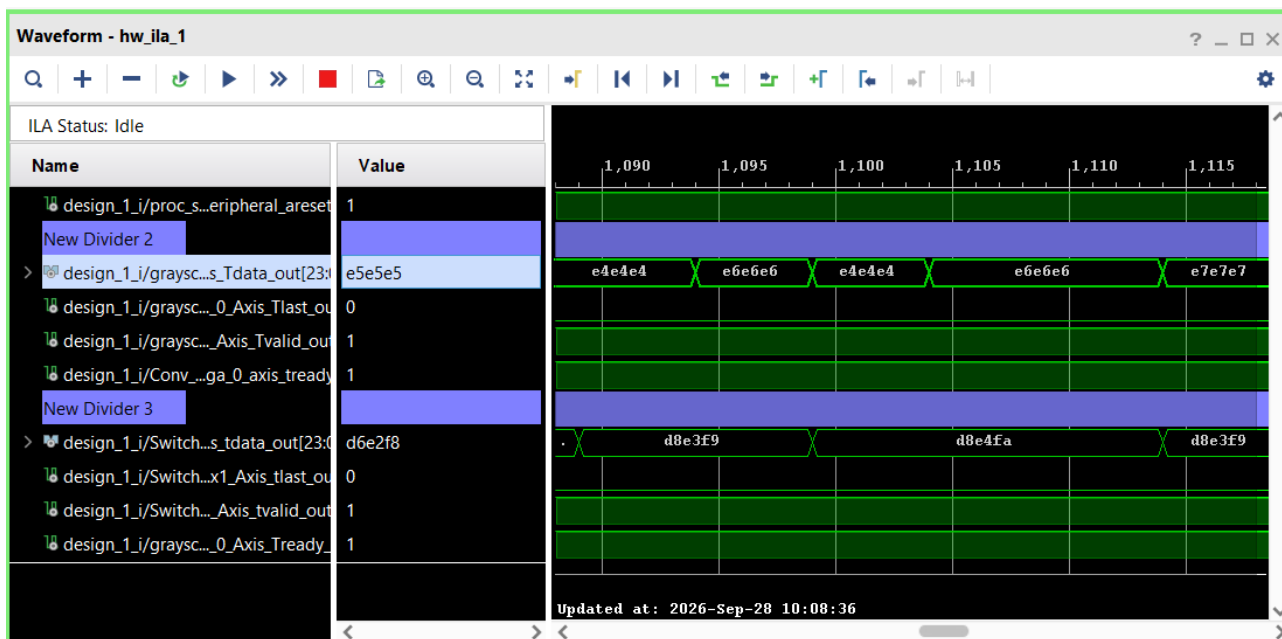
4.2.1 Test/démonstration à l'aide de l'ila et affichage sur l'écran.



Le bloc de conversion luminance est placé après le sélecteur de flux, il alimente le bloc de conversion Axis4Stream – VGA. Un ila est ajouté pour analyser les flux entrants et sortants du bloc.



Lorsque la bloc de conversion passe son signal Tready à 1, le signal se propage puis les transactions de données commencent.



Trigger sur Tvalid du bloc de sélecteur de flux et Tready du bloc de conversion Axi_VGA.

La sortie Tdata est bien constitué d'un octet répété, en revanche on observe une grosse désynchronisation entre Tdata in et out. Probablement lié au choix du domaine d'horloge.

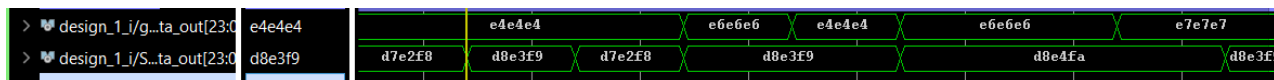


Illustration de la désynchronisation entre Tdata_in et Tdata_out.



Affichage de l'image en nuance de gris sur l'écran

3.2.3 Comparaison avec une golden reference.

Les scripts fournis par Deverne ne permettent pas d'analyser des images RGB mais le filtre réalise la somme de chaque canaux divisé par 3, on peut donc utiliser une image monocouleur fournit comme `sample_in_b.txt` par exemple. Puis diviser chaque valeur de pixel par trois pour obtenir le même résultat que le filtre.



Sortie du filtre grayscale avec `sample_in_b.txt` en entrée.



Imgj output de `sample_in_b / 3`.



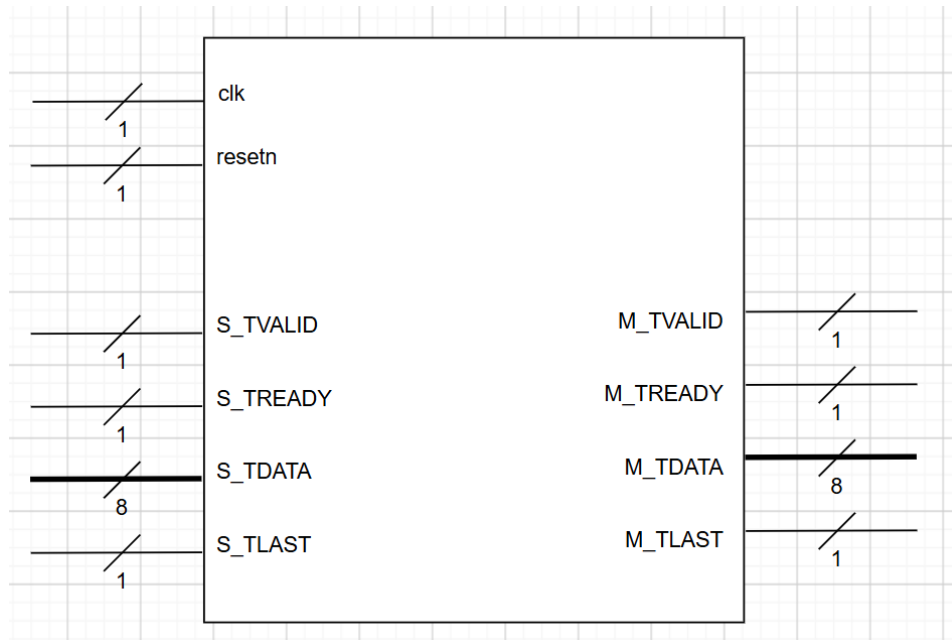
Résultat de la soustraction

Mean	Min	Max	
-0.335	-83	54.667	

Le résultat n'est pas parfaitement équivalent. Il est nécessaire d'investir cette différence pour établir si elle vient du filtre ou de la golden référence.

3.3 Filtre Gaussien

1/ Description fonctionnelle



Tout les blocs de cascade de filtres sont identiques.

Le filtre gaussien adopte une architecture en fenêtre glissante, il faudra instancier deux tableaux qui stockeront deux lignes ainsi que 9 registres afin d'appliquer les coefficients. Le retard induit entre l'entrée et la sortie sera significatif pour ce bloc.

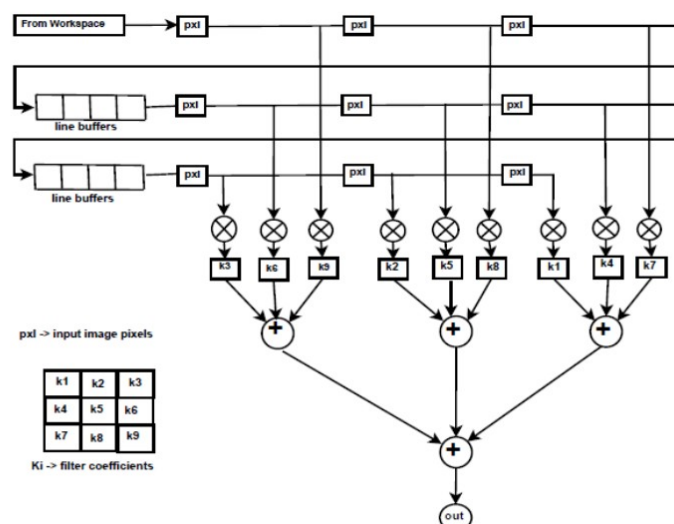


Fig. 3. Complete parallel hardware architecture of a 3x 3 filter kernel implementation for simplicity. Actually implemented 5x 5 kernel mask.

Design space exploration for image processing architectures on FPGA targets. Chandrajit Pal et al.

Le bloc sera validé, en simulation grâce à un premier testbench validant le comportement des signaux, puis nous utiliserons les scripts fournis par Deverne associés au filtre pour lire et écrire une image sous format texte.

Enfin le bloc sera ajouté à au design complet pour observer le résultat sur l'écran.

2/ Description des signaux internes et externes

Signaux Externes			
Nom du signal	Input/output	Type	Commentaire
C_coeff_[1..9]	Generic		Coefficients de la fenêtre glissante
clk	input	std_logic	Signal d'horloge
resetrn	input	std_logic	Signal de reset NOT
Axi4stream_in	Input	Axis4Stream	Flux axis4stream 8 bit
Axi4stream_out	Output	Axis4Stream	Flux axis4stream 8 bit

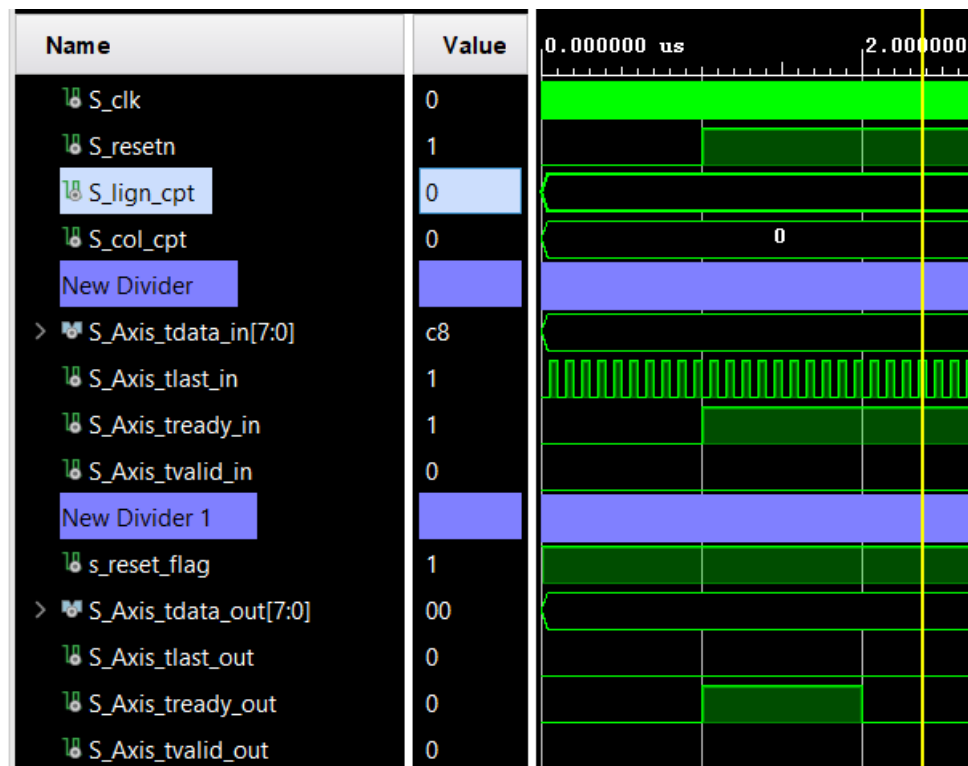
Signaux Internes		
Nom du signal	Type	Commentaire
SV_data_[1..9]	std_logic_vector(8bit)	Registre de stockage des pixels pour l'application de la fenêtre glissante.
SV_data_out	std_logic_vector(8bit)	Registre de stockage de la valeur final ramenée entre 0 et 255.
S_Axis_data_out_reg	std_logic_vector(8bit)	Registre de sortie du vecteur de donnée
S_Axis_data_in_reg	std_logic_vector(8bit)	Registre d'entrée du vecteur de donnée
S_Axis_T[..]_in_reg	std_logic	Registre d'entrée de la liaison Axi4stream
S_Axis_T[..]_out_reg	std_logic	Registre de sortie de la liaison Axi4stream
t_line_mem_[1..2]	Tableau de 640 octets	Registre pour le stockage de deux lignes.
S_col_cpt S_lign_cpt	Integer	Compteur de lignes et de colonnes.
s_reset_flag	std_logic	Flag permettant d'identifier que le système vient de démarrer, permet un traitement fin des signaux axis4stream.

3/ Test et validation

3.3.1 Modulation des signaux de la liaison AXIS4Stream.

Signal	Comportement attendu	Réussi ?
Resetn	Sur resetn actif, les compteurs internes sont réinitialisés, les registres de pixels ainsi que les tableaux.	
Axis_Tready_in	1 – Lorsque S_reset_flag est actif : Le système vient de démarrer, il est possible de charger la première ligne dans le tableau même si le bloc suivant n'est pas prêt. Si S_reset_flag = 1 et S_Tvalid_in = 1 alors Tready_in = 1. Lorsqu'on atteint la fin de la ligne. S_reset_flag tombe à 0. Le buffer de ligne est plein, le bloc ne peut admettre de nouvelles données que si le bloc suivant peut les accepter. C'est à dire si Tready_out vaut 1.	
Axis_Tvalid_out	À l'initialisation du système. Il ne faut pas délivrer la première ligne de l'image donc si S_reset_flag = 1 alors T_valid_out = 0. Ensuite, lorsque le compteur de ligne interne au bloc vaut 0, il faut débiter la dernière ligne de l'image précédente, mais si les données entrante ne sont plus correcte alors il faut interrompre la chaîne. Tvalid_out vaut 1 si Tvalid_in vaut 1, 0 sinon.	
Axis_TData_out	Diffusion des lignes : après une initialisation, Lorsque le compteur interne de ligne vaut 0, le bloc ne peut rien transmettre. Lorsque le compteur interne vaut 1, il faut émettre la première ligne Stocké dans le buffer. Une fois que la première image à été diffusée, Axis_Tdata_out à la ligne 0 soit valoir SV_data_5, Diffusion des colonnes ; Lorsque le compteur interne vaut 0 et 1, La valeur de Axis_Tdata_out doit valoir SV_data_5, Le produit de la convolution sinon.	
Axis_TLast_out	Tlast out vaut 1 lorsque le compteur de colonne vaut 1 sauf à l'initialisation lors S_reset_flag vaut 1.	

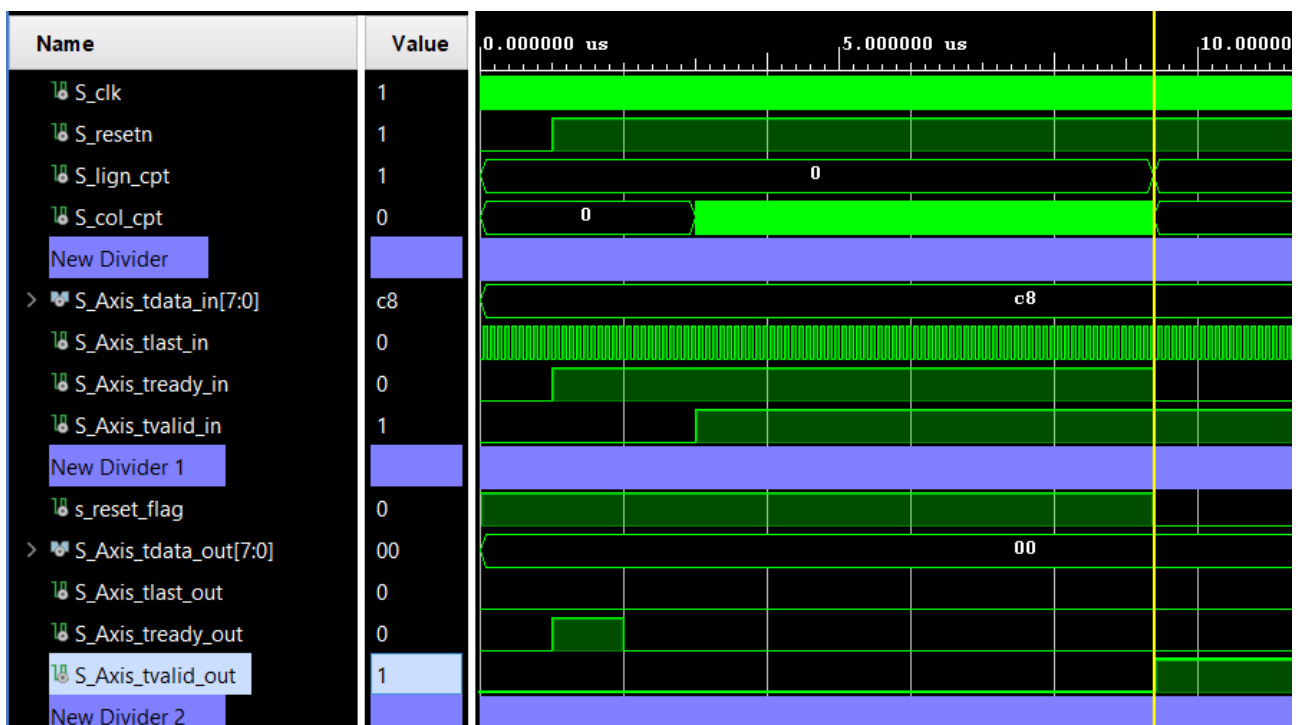
a) Le bloc aval demande des données avant que le filtre gaussien n'ai chargé la première ligne.



Au commencement de la simulation, le buffer de la première ligne n'est pas chargé. Le bloc de filtrage demande des données avec Tready_in actif mais Tvalid_out reste à l'état bas tant que le bloc n'a pas rempli son buffer.

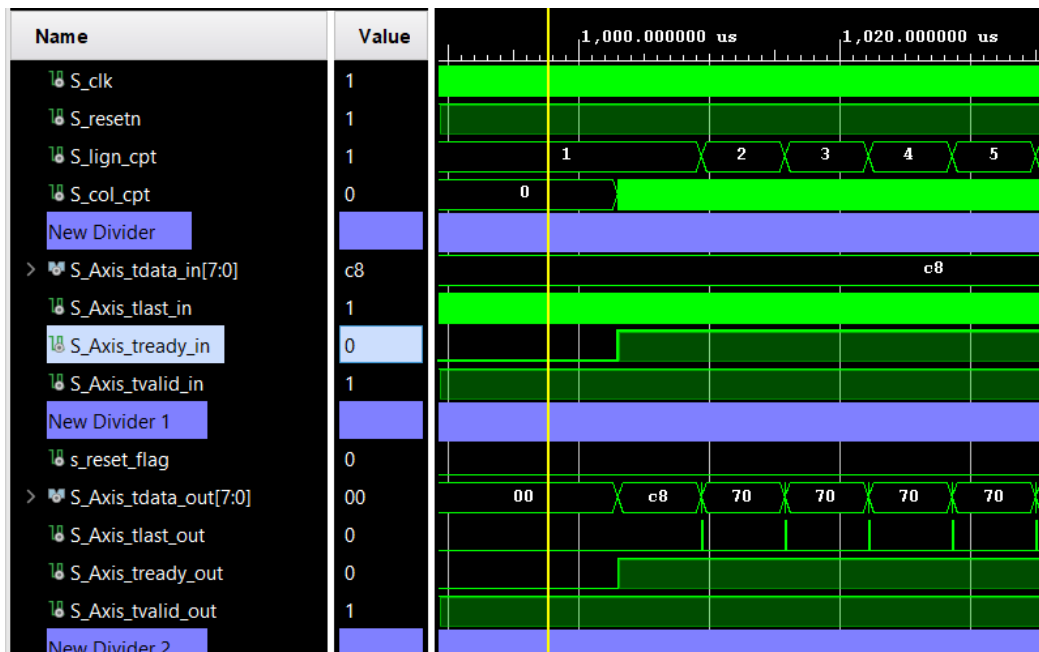
Lorsque Tready_out monte, aucune donnée n'est débitée, les compteurs internes restent inertes.

b) Chargement de la première ligne de l'image



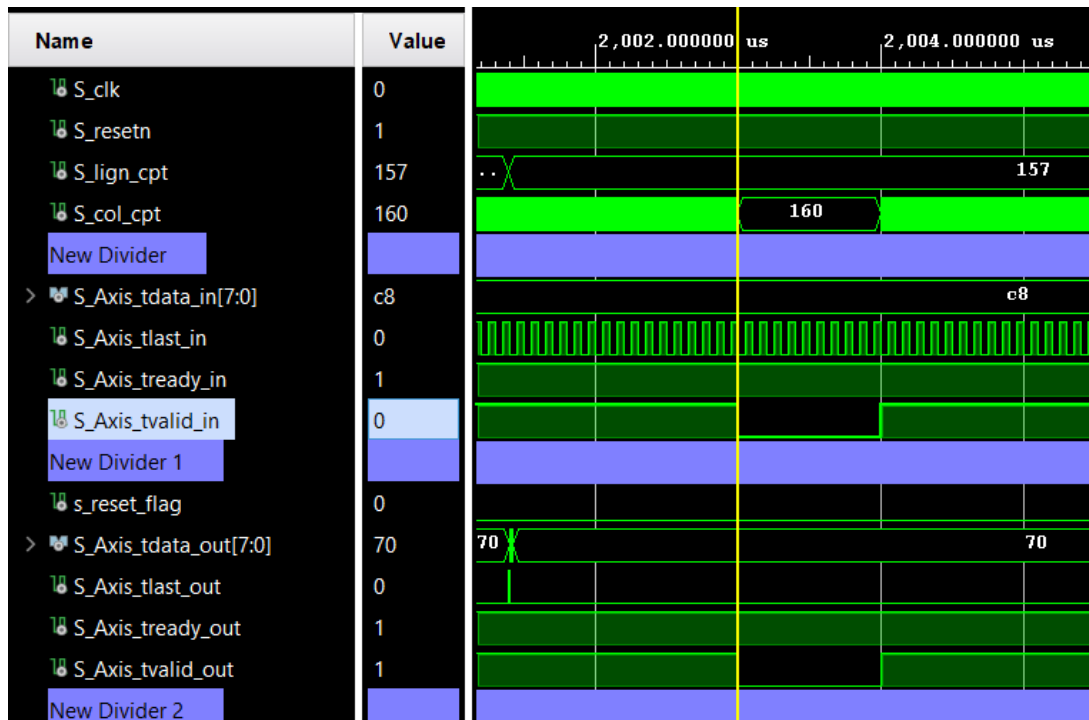
Tready_in est à l'état haut, sur Tvalid_in à l'état haut les compteurs défilent et le filtre remplit son buffer. Un fois plein, tready_in devient inactif et le bloc signale qu'il est prêt à débiter avec Tvalid_out actif.

c) fonctionnement nominal



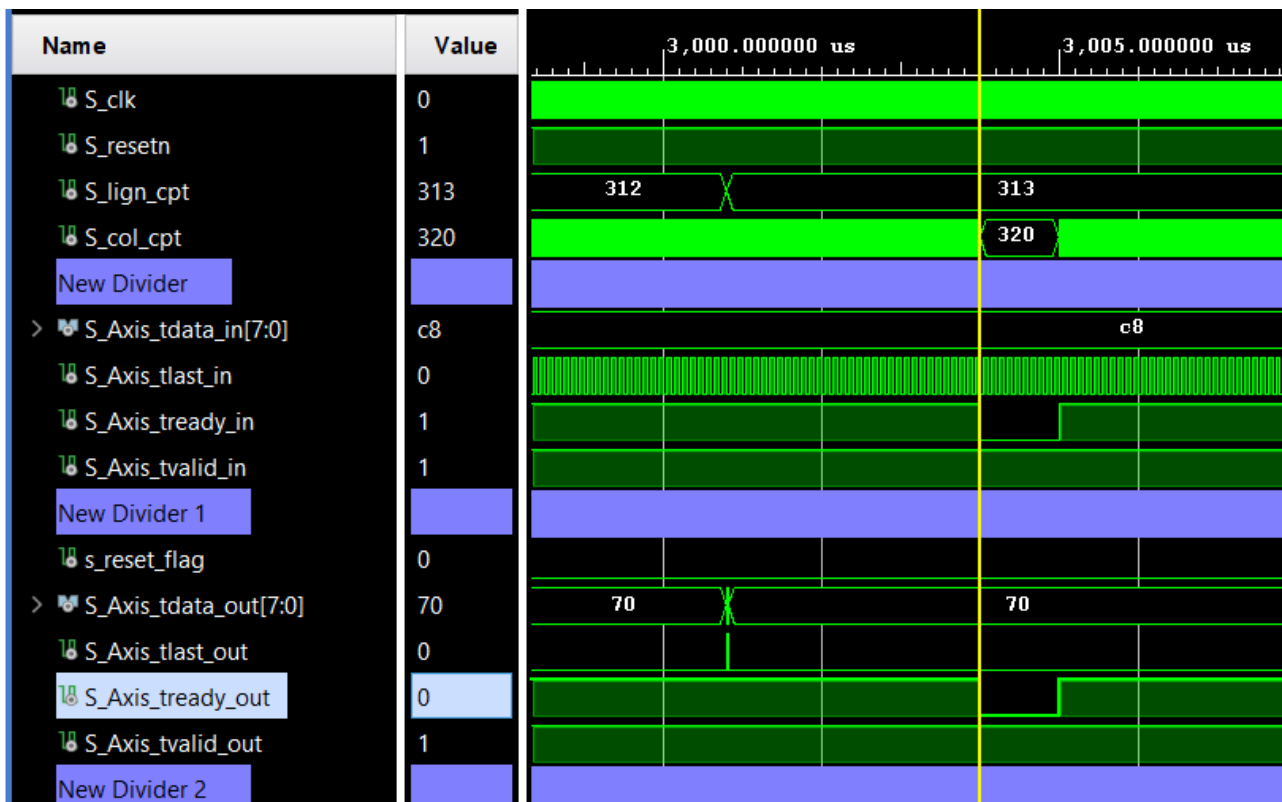
Lorsque T_ready_in est actif, le filtre débite la première ligne stocké dans son buffer. Tdata_out vaut c8 pendant toute la ligne ce qui indique qu'aucune transformation n'est appliquée. Tlast_out s'active une fois que la ligne à été débitée. Le compteur interne compte les lignes et les colonnes.

c) Le bloc amont n'a plus de données valides.



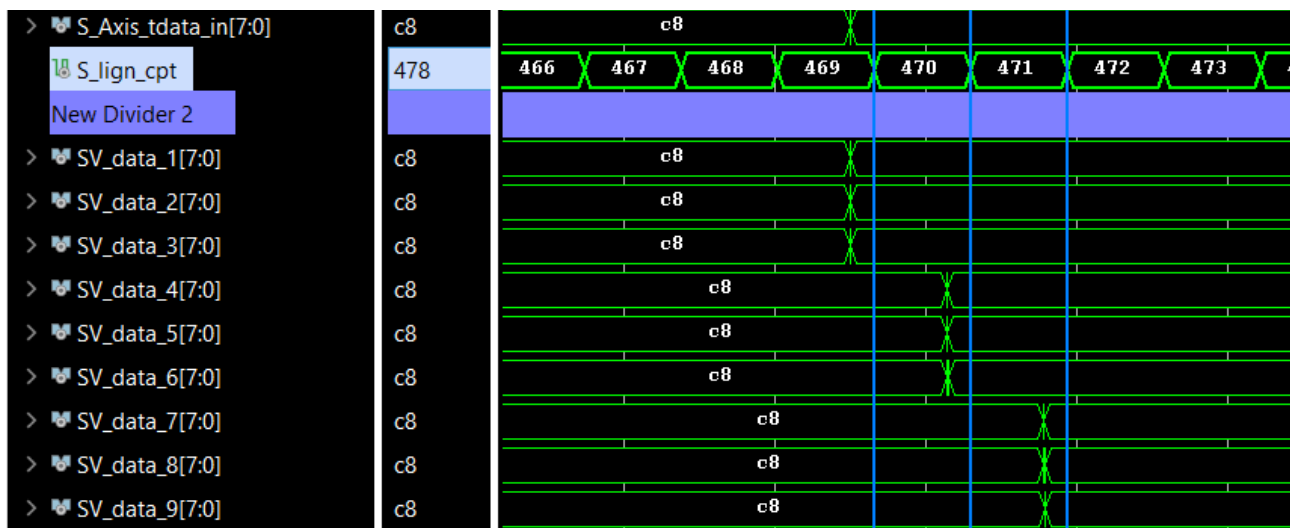
Lorsque T_valid_in est inactif, Tvalid_out devient inactif également. Le robinet se ferme.

d) Le bloc aval n'est plus capable de recevoir les données.



Lorsque Tready_out est inactif, Tready_in le devient également. C'est le feu rouge !

e) Vérification du bon fonctionnement des buffers et de la fenêtre glissante.



Ce diagramme met en évidence la propagation d'une donnée dans la fenêtre glissante et dans les buffers. Le flux de données est mis égal à « 11 » pendant un coup d'horloge, cette donnée est propagée dans les registres 1,2 et 3. puis dans la ligne suivante dans les registres 4,5 et 6 et enfin dans les registres 7,8 et 9.

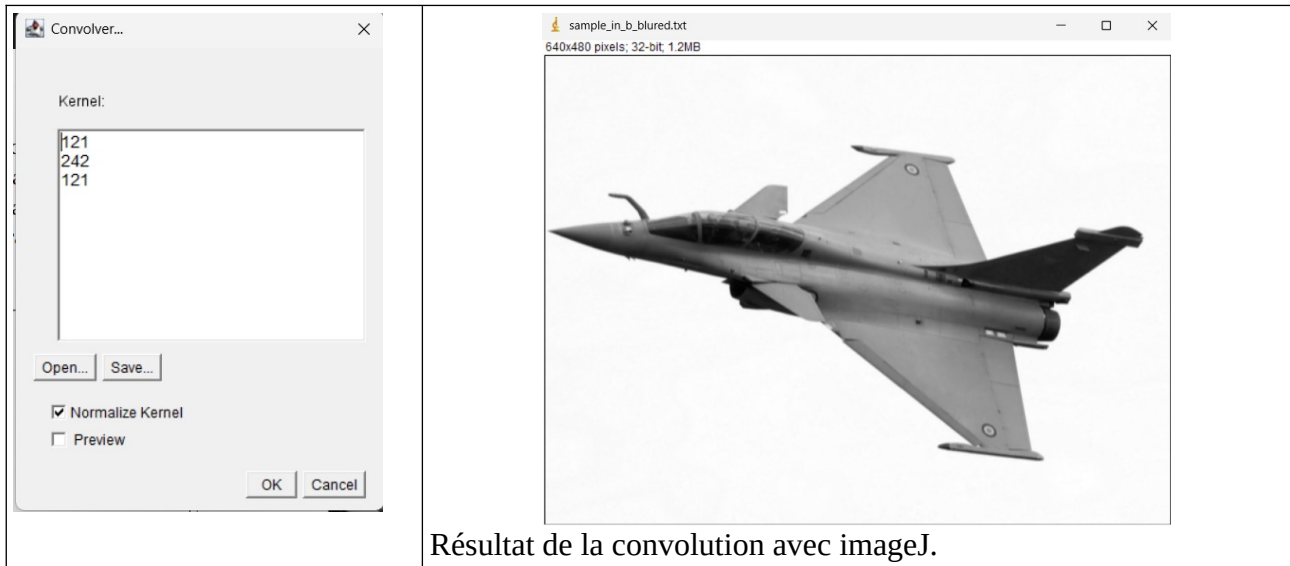
3.3.2 Comparaison avec la golden reference : Utilisation des modules fournis par deverne.



L'image a bien été floutée.

Pour s'assurer que les valeurs calculées sont correctes, il est nécessaire d'utiliser un outils dédié dont nous sommes certains de sa justesse. ImageJ sera utilisé pour appliquer la convolution avec les

coefficients choisis puis l'image floutée obtenue sera soustraite à l'image de sortie du filtre. Le test est réussi si cette soustraction permet d'obtenir une image noire car la valeur des pixels sera nulle.

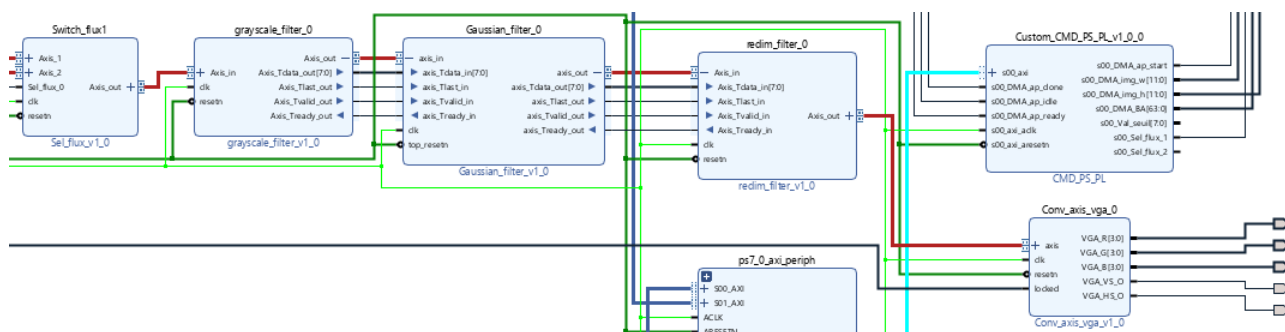


Résultat de la soustraction entre stream_out et l'image convolutive calculée avec ImageJ.

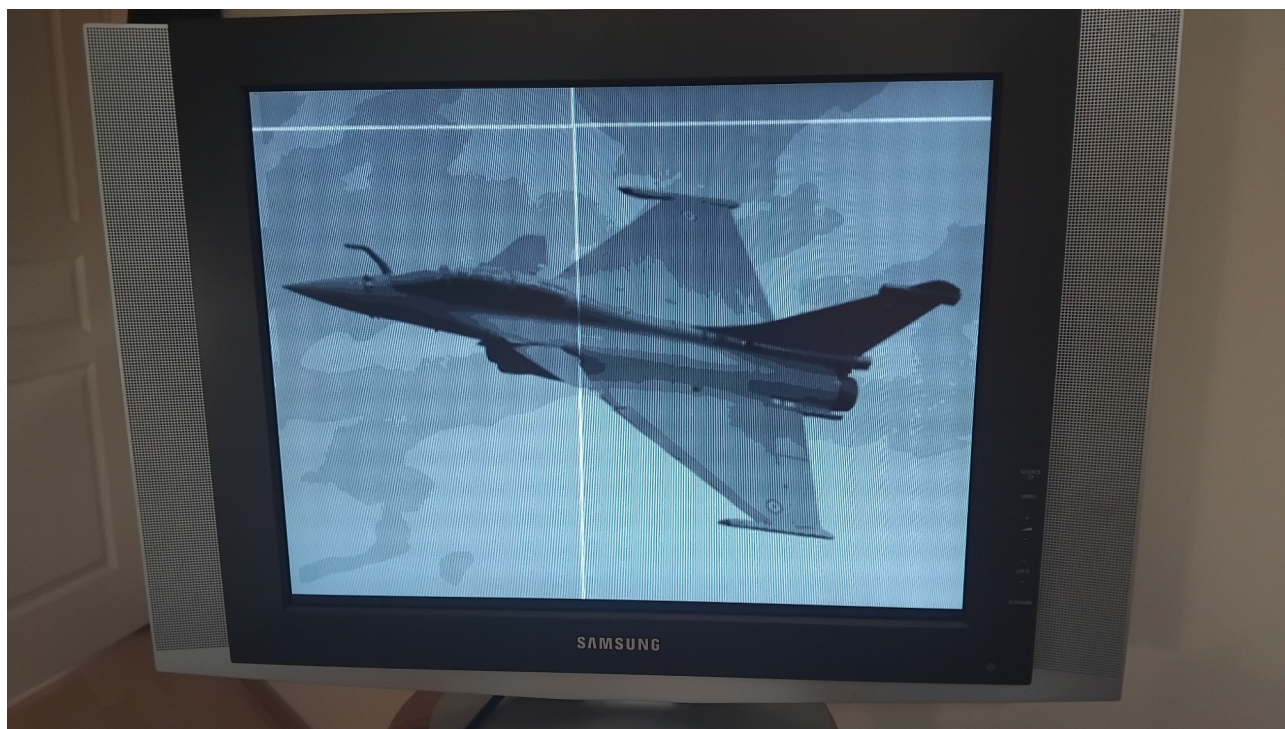
L'image n'est pas uniforme, impliquant que le traitement appliqué par les deux filtres ne sont pas les mêmes. Il sera nécessaire d'investir cette différence.

3.3.3 Test/Démonstration : intégration du bloc dans le projet global.

Un bloc de redimensionnement du flux de données est rajoutée pour remettre le vecteur de données du 24 bits admissible par le convertisseur Axis-VGA. Ce bloc est optionnel car nous pourrions connecter la sortie du bloc de flou gaussien directement en entrée du bloc VGA. Nous aurions une image en nuance de rouge plutôt qu'en nuance de gris.



L'image obtenue sur l'écran est la suivante :



L'image est stable et le filtre à été appliqué mais elle est bardée d'artefacts qui se déplacent sur l'écran dont la cause reste à déterminer.

1/ Schéma synoptique global

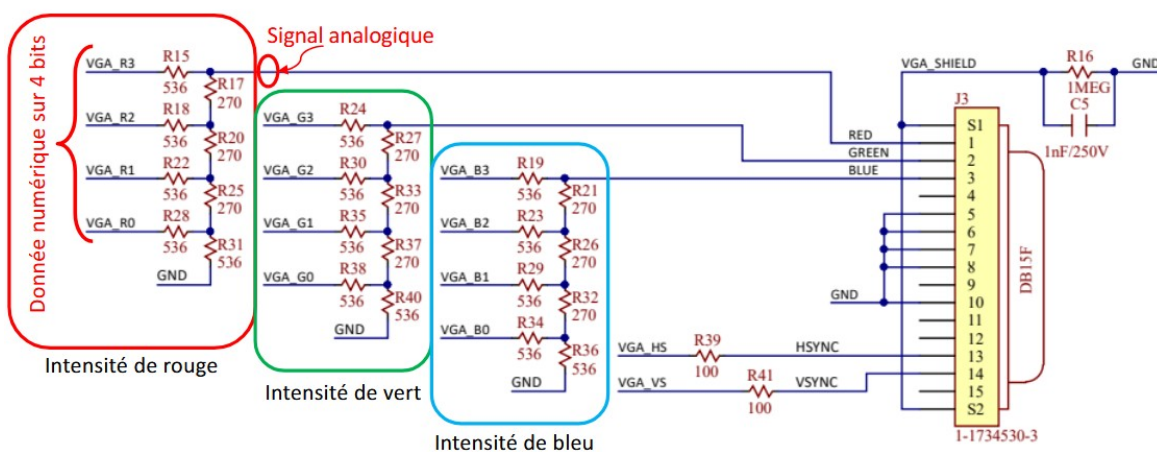
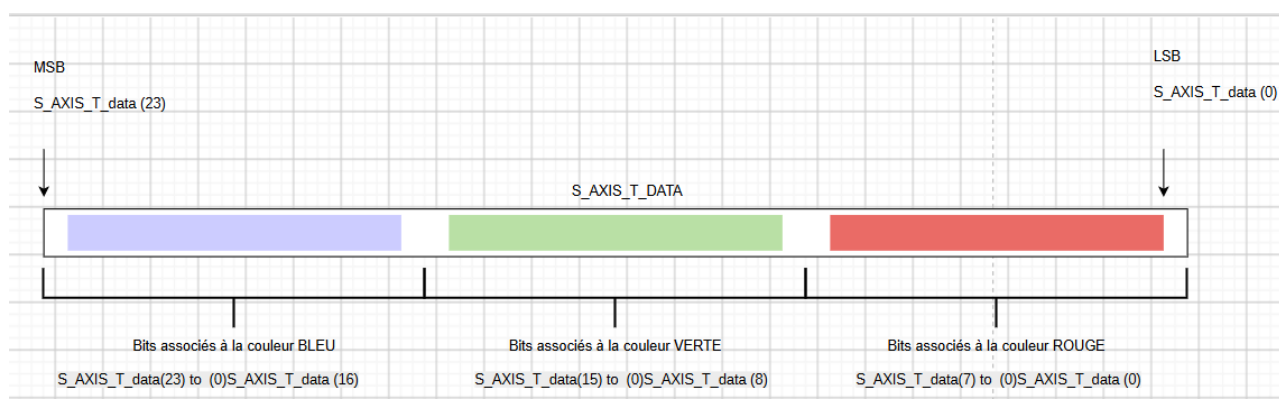
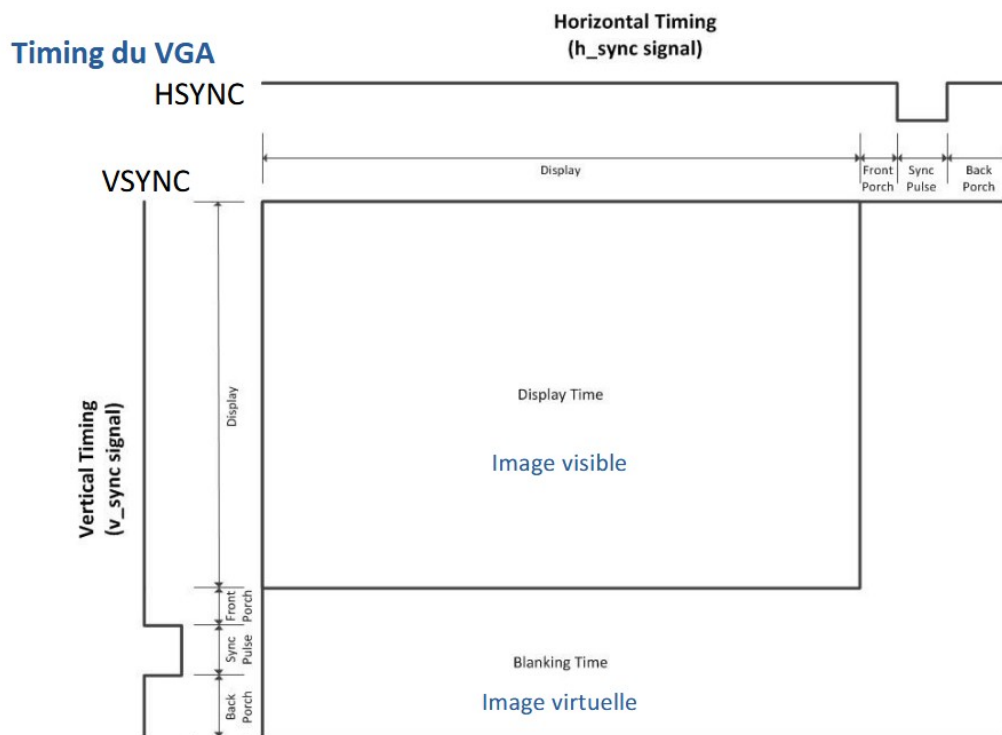


Schéma issu de la documentation du Pmod VGA

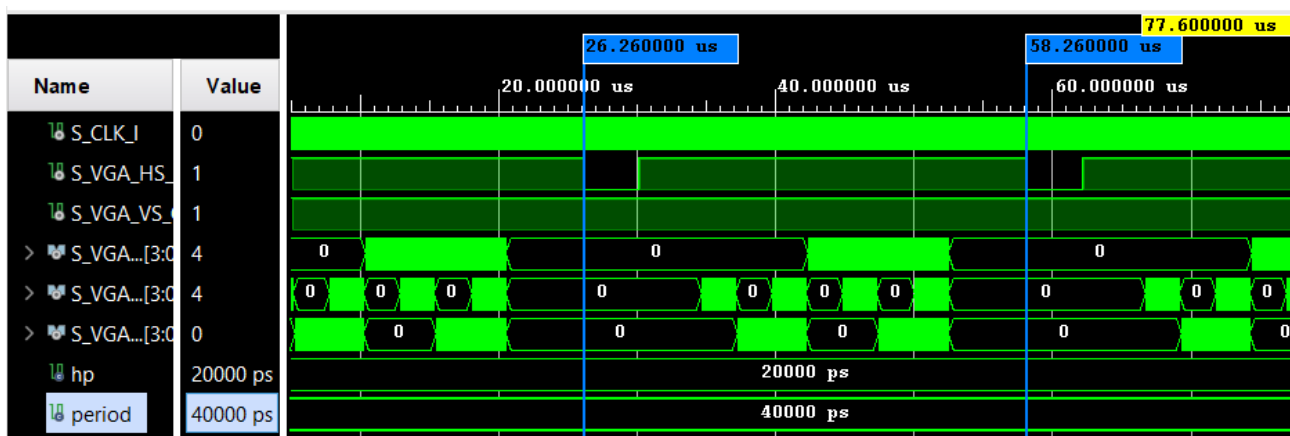
16

S_AXIS_T_data = vector(24bit)

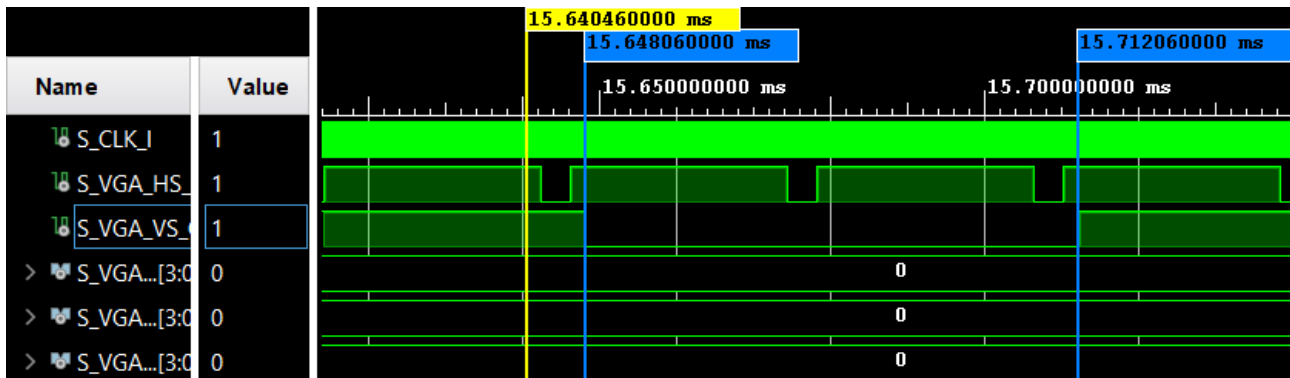




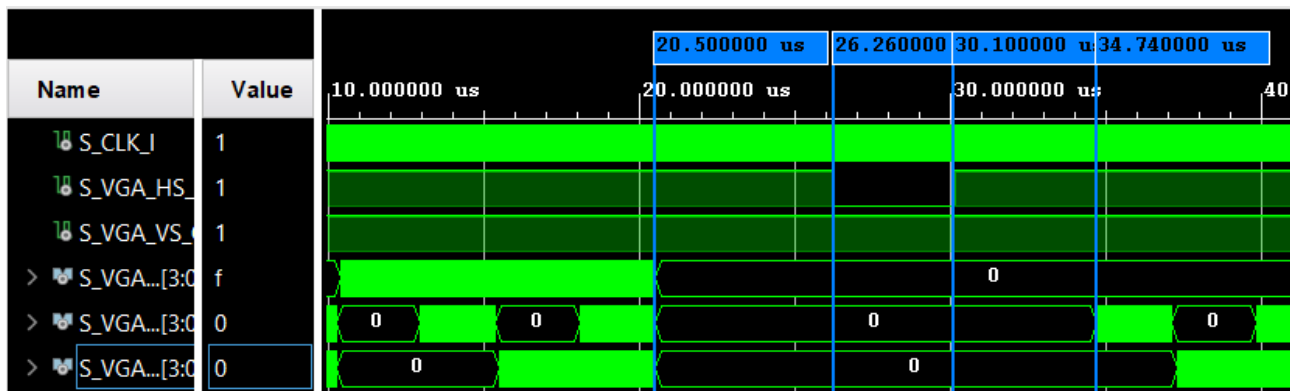
Mesures des timing du référence design fourni avec l'adaptateur pmod VGA.



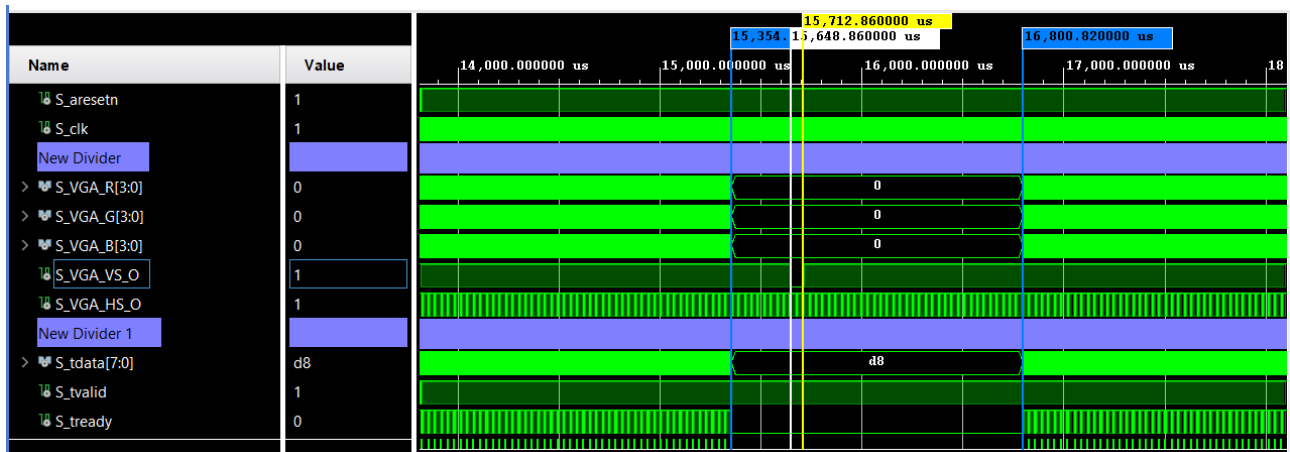
Fréquence Hsync



Pulse Width Vertical Synchronisation



Horizontal front and back porch



Vertical front and back porch

		Explication
Frame Width	640	
Frame Height	480	
H_max	800 ?	--H total period (pixels)
H_FP	16	H front porch

H_BP	208	$H_{back\ porch} = H_{max} - H_{pw} - H_{fp} - Frame_height = 800 - 96 - 16 - 480$
H_PW	96	H_sync pulse width (pixels)
V_FP	10	V front porch --(lines)
V_backport	33	$V_{back\ porch} = V_{max} - V_{FP} - V_{PW} - Frame\ height$
V_PW	2	V sync pulse width (lines)
V_max	525	Vertical total period (lines) ?
H_POL	0	Pulse polarity.
V_POL	0	

Nom	Width required	Delais calculé @25Mhz	Délais mesuré
V_sync pulse width	2*800	64 us	64 us
V_sync frequency	(480+10)*800	15,6 ms	16,8 ms
H_sync pulse width	96	3,84 us	3,84 us
H_sync frequency	800	32 us	32 us
H front porch	16	0,64 us	1,92 us
H Back porch	208	8,32 us	6,1 us
V front porch	10*800	320 us	295 us
V Back porch	33*800	1056 us	1091 us

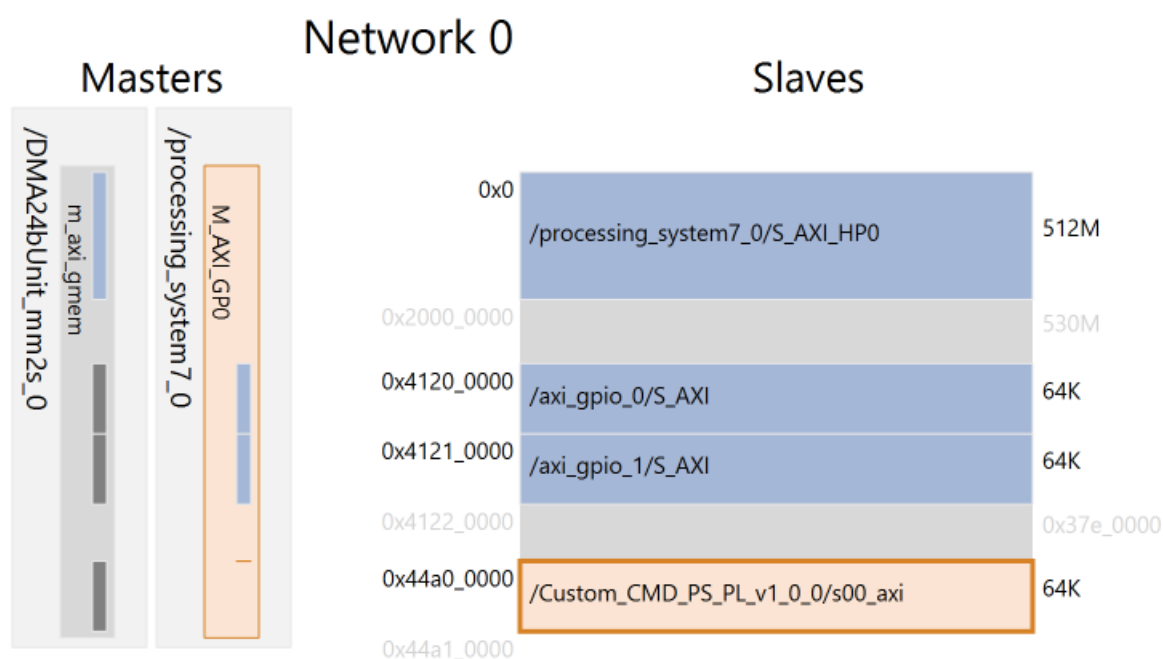
Hypothèse : il y a des 0 qui sont envoyés en fin ou en début d'écran. Ce qui permet pas de calculer correctement le front et le back porch.

3/ note sur la mémoire

Chaque adresse mémoire correspond à 1 octet. De 0x0000 à 0x0004, il y a 4 octets donc 32 bits.

En hexadécimal, 2 digits correspondent à 1 octet. Donc 32 bits = 0x0000_0000

La taille des 10 registres du DMA est de 32 bits, l'offset de l'adresse vaut 0x04 à chaque fois. La base adresse de ces registres est choisie par vivado dans l'onglet adress map :



La base adresse des registres du bloc de commande est indiqué ci-dessus : 0x44a0_0000

Name	Interface	Slave Segment	Master Base Address	Range	Master High Address
Network 0					
/DMA24bUnit_mm2s_0					
/DMA24bUnit_mm2s_0/Data_m_axi_gmem (64 address bits : 16E)					
/processing_system7_0/S_AXI_HP0	S_AXI_HP0	HP0_DDR_LOWOCM	0x0000_0000_0000_0000	512M	0x0000_0000_1FFF_FFFF
Excluded (3)					
/processing_system7_0					
/processing_system7_0/Data (32 address bits : 0x40000000 [1G])					
/axi_gpio_0/S_AXI	S_AXI	Reg	0x4120_0000	64K	0x4120_FFFF
/axi_gpio_1/S_AXI	S_AXI	Reg	0x4121_0000	64K	0x4121_FFFF
/Custom_CMD_PS_PL_v1_0/s00_axi	s00_axi	reg0	0x44A0_0000	128	0x44A0_007F
Excluded (1)					
/processing_system7_0/S_AXI_HP0	S_AXI_HP0	HP0_DDR_LOWOCM			

La base adresse du DMA vaut : 0x44a0-0000, la plage est fixée au minimum possible 128 octets.

La sélection du registre correspondant s'effectue en décodant l'adresse dans le bloc de commande PS-PL. Ce décodage correspond à l'extraction des bits 2 à 5.

Reg_Add hex	2 derniers octets en binaire Loc_add from Custom_CMD_PS_PL (5 down to 2)	Registre correspondant	Valeur initiale
0x44a0-0000	0000-0000	DMA_Start	0x0000_0000
0x44a0-0004	0000-0100	IMG_W	0x0000_0280
0x44a0-0008	0000-1000	IMG_H	0x0000_01E0
0x44a0-000C	0000-1100	IMG_BA_LSB	0x0040_0000
0x44a0-0010	0001-0000	IMG_BA_MSB registre toujours égal à 0	0x0000_0000
0x44a0-0014	0001-0100	VAL_SEUIL	0x0000_00C8
0x44a0-0018	0001-1000	SEL_FLUX_1	0x0000_0000
0x44a0-001C	0001-1100	SEL_FLUX_2	0x0000_0000
0x44a0-0020	0010-0000	DMA_INPUT	NA

Il faut fournir à L'entrée image_in du DMA la base adresse de l'image située dans la mémoire DDR. Pour accéder à la plage mémoire du DDR, il faut aller la chercher dans xparameter.h :

```
/* Definitions for peripheral PS7_DDR_0 */
#define XPAR_PS7_DDR_0_S_AXI_BASEADDR 0x00100000
#define XPAR_PS7_DDR_0_S_AXI_HIGHADDR 0xFFFFFFFF
```

La mémoire DDR contient également le programme C, il est nécessaire pour ne pas écraser le programme lorsqu'on charge l'image d'aménager un espace. On prendra comme base adresse de l'image l'adresse suivante : 0x0040_0000. Cette adresse tient en 32 bits, néanmoins comme l'entrée image_in du DDR est en 64 bits, on va fixer les MSB de image_in à 0. (IMG_BA_MSB est toujours égal à 0).

Pour charger l'image dans la mémoire DDR à l'aide de la console XCST on va utiliser la ligne de code suivante :

```
mwr -bin -file "C:/Users/cleme/Desktop/POEI/Cours/Projet_Corner_detect/Vitis_WD/PS_prog/sample_in_rgb.bin" 0x00400000 307200
```

0x0040_0000 est l'adresse de base où l'image va être enregistrée et 307200 correspond au nombre de valeurs à écrire dans la mémoire, c'est le nombre de pixel car on stocke un pixel RGB par élément de mémoire (On a besoin de 3 octets).

4/ Historique de résolution de divers problèmes rencontrés.

Problème rencontré	Résolution
Pas d'affichage sur le port UART	changement du port de sortie pour stdin et stdout dans le fichier .spr de configuration de la plateforme sur Vitis pour ps7_uart0.
Problème de dialogue avec la mémoire DDR, L'application sur vitis plante directement lors du launch debug	Mauvaise configuration du DDR Importation de la configuration du PS du reference design fourni

Problèmes à résoudre :

1/ la fonction de lecture du fichier de fonctionne pas

<https://xilinx-wiki.atlassian.net/wiki/spaces/A/pages/18841771/xilffs>

le fopen ne marche pas en embedded. Il faut investir le sujet ci-dessus.

2/ Créer un pattern de debug sur l'écran à l'aide de la console.

3/ Pour éviter de décapiter le programme, il est souhaitable de déclarer un pointeur Uint32 vers un tableau 640*480 et passer le pointer via l'espace registre. Au lieu de déclarer l'adresse en DUR.